Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University

THE EVOLUTION OF MACHINE LEARNING AND AI

Week 5 Detailed Booklet

Science-Inspired AI: Building Physical Law Into Models

Award	Certificate in Artificial Intelligence and Machine Learning Engineering (30 ECTS)
Parent programme	Higher Diploma in Science in Computer Science (NFQ Level 8)
Module	The Evolution of Machine Learning and AI (5 ECTS, Semester 1)
Week	Week 5 of 12 · Sprint 2: Architecture as inductive bias, then the science frontier
Indicative content	Science-grounded models: physics-informed networks, symmetry, neural ODEs, neural operators and diffusion
Learning outcomes	LO2: Characterise the training of model families, including diffusion and science-grounded models
Assessed in	Mid-module quiz, Week 6 (50% of module mark)
Companions	PINN Sandbox · Symmetry Explorer · Diffusion and Thermodynamics Explorer · Lab: six exercises in numpy, no PyTorch

Read this first. This is the **detailed** account of Week 5, kept for students who want to go deeper. **You are not required to read it.** The required reading is the short **Core Booklet**, and the Week 6 quiz can only ask what the Core Booklet's red boxes say. The **Reading Map** on the next page marks every chapter CORE (also covered, more briefly, in the Core Booklet) or EXTRA (optional). Boxes marked **In Plain English** restate an idea with the jargon removed; if a section loses you, skip to that box.

Contents

1 The Strongest Prior There Is . . . 5
1.1 The difference between a regularity and a law . . . 5
1.2 The result that sets up the week . . . 5
1.3 The six ideas of the week . . . 6
2 What "Physics-Informed" Actually Means . . . 8
2.1 The loss, written out . . . 8
2.2 What is the same, and what is different . . . 9
3 Reading a Differential Equation . . . 11
3.1 The three symbols . . . 11
3.2 Reading the oscillator equation as a sentence . . . 11
3.3 Why the conditions are separate . . . 12
4 Where Scientific Knowledge Lives . . . 13
4.1 Why the last two are underrated . . . 13
5 The Data Regime That Makes This Pay . . . 15
5.1 Where the payoff is largest . . . 15
6 Reading the Notation of This Week . . . 17
7 Differentiating a Derivative . . . 19
7.1 The problem . . . 19
7.2 Why not just use finite differences . . . 20
8 The PINN, End to End . . . 22
8.1 The complete recipe . . . 22
8.2 The four results . . . 22
9 Choosing the Network . . . 24
9.1 Why ReLU is disqualified . . . 24
9.2 Input scaling, which matters more here . . . 24
9.3 Depth, width and the hard part . . . 25
10 Hard Constraints, Soft Penalties, and a 400-Fold Difference . . . 26
10.1 The soft way, and its problem . . . 26
10.2 The hard way . . . 27
11 Collocation Points, and the Loss You Are Really Minimising . . . 29
11.1 They cost nothing, and that is the point . . . 29
11.2 How many, and where . . . 29
12 What You Get For Free . . . 31
12.1 The experiment . . . 31
12.2 A note on the middle row . . . 32
13 From an ODE to a PDE . . . 33
13.1 The problem . . . 33
13.2 The three changes . . . 33
14 Symmetry: The Prior That Is Exactly True . . . 35
14.1 The task . . . 35
14.2 The two models . . . 36
15 Equivariance in Practice, and When It Is Wrong . . . 38
15.1 How this is done properly . . . 38
15.2 The counterexample . . . 38
16 Neural ODEs: Learning the Rule, Not the Answer . . . 40
16.1 The inversion . . . 40
16.2 How it is trained . . . 40

16.3 What it buys . . . 41
17 Structured Dynamics: Learning Only the Unknown Half . . . 42
18 Neural Operators: Solving a Whole Family at Once . . . 44
18.1 The limitation being fixed . . . 44
18.2 DeepONet, which is two networks and a dot product . . . 44
19 The Operator Landscape . . . 46
19.1 The Fourier Neural Operator . . . 46
19.2 Choosing between them . . . 46
20 Diffusion Models as Non-Equilibrium Thermodynamics . . . 48
20.1 The physical picture . . . 48
20.2 Why the structure makes the learning easy . . . 49
21 The Score, and What Was Borrowed . . . 51
21.1 The score . . . 51
21.2 The connection to the training objective . . . 51
21.3 The vocabulary that came from physics . . . 52
22 Thermodynamics-Informed Models . . . 53
22.1 The GENERIC structure, and thermodynamics-informed GNNs . . . 53
23 When Physics-Informed Fails . . . 55
23.1 A wrong equation . . . 55
23.2 The four ways this goes wrong . . . 56
23.3 How to check . . . 56
24 Choosing a Science-Grounded Method . . . 58
24.1 The three questions . . . 58
25 Responsibility: Models That Cannot Lie About Physics . . . 60
25.1 The guarantee you can actually make . . . 60
26 A Field Guide: Seven Questions . . . 62
27 The Revision Core: The Whole Week on Two Pages . . . 63
27.1 The one sentence . . . 63
27.2 The six methods . . . 63
27.3 The numbers to know . . . 63
27.4 PINNs . . . 64
27.5 What you get free . . . 64
27.6 Symmetry . . . 64
27.7 Learning rules rather than answers . . . 64
27.8 Diffusion . . . 64
27.9 Ten one-line lessons . . . 64
28 Glossary in Plain English . . . 66
29 Question Bank . . . 68
29.1 Short answer: two to three sentences each . . . 68
29.2 Applied: read the numbers, do the reasoning . . . 69
29.3 Discussion prompts . . . 69
30 Further Reading . . . 71
Appendix: How Week 5 Maps to the Module . . . 72

Reading map: what is core and what is extra

CORE means the idea is examinable and the Core Booklet already covers it in plain English; read the chapter here only for the fuller version. **EXTRA** means optional: interesting, not examined, safe to skip.

Ch.	Title	Status
1	The Strongest Prior There Is	CORE
2	What “Physics-Informed” Actually Means	CORE
3	Reading a Differential Equation	CORE
4	Where Scientific Knowledge Lives	EXTRA
5	The Data Regime That Makes This Pay	CORE
6	Reading the Notation of This Week	EXTRA
7	Differentiating a Derivative	CORE
8	The PINN, End to End	CORE
9	Choosing the Network	EXTRA
10	Hard Constraints, Soft Penalties, and a 400-Fold Difference	CORE
11	Collocation Points, and the Loss You Are Really Minimising	CORE
12	What You Get For Free	CORE
13	From an ODE to a PDE	EXTRA
14	Symmetry: The Prior That Is Exactly True	CORE
15	Equivariance in Practice, and When It Is Wrong	EXTRA
16	Neural ODEs: Learning the Rule, Not the Answer	CORE
17	Structured Dynamics: Learning Only the Unknown Half	EXTRA
18	Neural Operators: Solving a Whole Family at Once	CORE
19	The Operator Landscape	EXTRA
20	Diffusion Models as Non-Equilibrium Thermodynamics	CORE
21	The Score, and What Was Borrowed	EXTRA
22	Thermodynamics-Informed Models	EXTRA
23	When Physics-Informed Fails	CORE
24	Choosing a Science-Grounded Method	CORE
25	Responsibility: Models That Cannot Lie About Physics	EXTRA
26	A Field Guide: Seven Questions	CORE
27	The Revision Core: The Whole Week on Two Pages	CORE
28	Glossary in Plain English	CORE
29	Question Bank	EXTRA
30	Further Reading	EXTRA

Inside a CORE chapter, the boxes marked ⌕ Deeper Detail and ✎ Worked Example are themselves extra. The examinable statements are exactly the red ⋆ boxes in the Core Booklet; nothing here adds to that list.

1 The Strongest Prior There Is

COVERS	Why a law of nature is a different kind of assumption from anything in Week 4
TIME	Read first, about 20 minutes
IN BRIEF	Week 4 built in assumptions that were approximately true and got 128 times the data for them. This week builds in assumptions that are exactly true, and the payoff is larger and stranger.

Week 4 ended on a bargain. Structure built into a model substitutes for data: when your assumptions match reality you need fewer examples, and when they do not you get a model that is worse than one which assumed nothing. The convolution's locality assumption was worth roughly 128 times the data on that week's task.

Now notice something about that assumption. **Meaning is local** is a decent generalisation about photographs. It is not a fact. There are images where it fails, there are datasets where it misleads, and the constant "roughly" in front of every claim about it is doing real work.

This week asks what happens when the assumption is not a decent generalisation but a law.

1.1 The difference between a regularity and a law

KEY IDEA

A statistical regularity is a pattern that usually holds in the data you happen to have. A physical law is a constraint that holds **always, everywhere, exactly**, whether or not you have data, whether or not your data is representative, and whether or not anybody has measured the case in front of you.

Nothing else you build into a model has that property. A convolution's locality can be violated by an unusual image. Conservation of energy cannot be violated by an unusual anything.

That difference is not rhetorical. It changes what you can do with the assumption.

- A regularity gives you a **prior**: a nudge towards answers that have usually been right.
- A law gives you a **constraint**: a region of answer-space that is simply excluded, and the model is not permitted to leave it no matter what the data says.

IN PLAIN ENGLISH

Imagine two kinds of advice while you are trying to work out a stranger's job from a photograph.

The first kind: **people in high-visibility jackets are often construction workers.** Useful, frequently right, occasionally wrong, and it does not eliminate anything. You still have to consider every possibility.

The second kind: **this person is standing in Dublin, so they are not currently in Cork.** That is not a hint. It removes an entire set of answers permanently, and no amount of new evidence about the jacket will bring them back.

Week 4 gave models the first kind of advice. This week gives them the second, and the second kind is enormously more powerful when you can get it.

1.2 The result that sets up the week

The lab builds a physics-informed neural network for a mass on a spring with friction. It is compared against an identical network trained the ordinary way, on measurements.

MEASURED IN THE LAB

From `experiment_data_efficiency()` in `pinn.py`. Both models are the same network: 2,209 parameters, the same initialisation, the same optimiser, the same number of epochs. **Only the loss differs.**

Model	Observations used	Error against the true solution
physics-informed	**0**	**0.00135**
data-only	4	0.31289
data-only	8	0.06556
data-only	16	0.04326
data-only	32	0.02962
data-only	64	0.02648

Read the top row's second column again. It is zero. The physics-informed model never sees a single measurement of the thing it is predicting, and it is **twenty times more accurate** than a model given sixty-four of them.

This is not a better model in the ordinary sense. It is a model that was told what the answer must satisfy instead of being shown examples of it.

WHY THIS MATTERS

The reason this matters practically, and not just as a curiosity, is the regime almost everyone actually works in.

Week 4's Chapter 21 pointed out that the Vision Transformer beats a convolutional network given 300 million images and loses given 1.3 million. Most people reading this will never have 1.3 million labelled examples of anything. You will have forty measurements from one turbine, or two hundred patients from one hospital, or a fortnight of readings from one sensor.

In that regime, more data is not available at any price, and the only lever left is **what you already know**. Science and engineering happen to come with a great deal of extremely reliable prior knowledge, written down over four centuries, and until recently machine learning made almost no use of it.

That is what this week is about: the machinery for putting known physics inside a model, and an honest account of when it helps and when it does not.

1.3 The six ideas of the week

They look like six separate methods and they are one idea applied to six different pieces of known structure.

Method	What piece of knowledge it builds in	Chapter
Physics-informed networks	the governing equation itself, checked wherever you like	8 to 12
Neural operators	that you want a whole family of solutions, not one	18, 19
Neural ODEs	that the world evolves continuously, by a rule that does not change	16, 17
Equivariant models	an exact symmetry: rotation, translation, permutation	14, 15

Diffusion models	a known physical process, used as the scaffolding for a hard problem	20, 21
Thermodynamics-informed models	that certain quantities can only ever increase or stay constant	22

Where this came from, briefly

The idea is older than it looks. Fitting differential equations with neural networks was tried in the 1990s, by Lagaris and colleagues among others, and it worked on small problems and then went quiet, because the automatic differentiation and the compute were not there.

It returned in 2017 and 2019 with Raissi, Perdikaris and Karniadakis, who gave it the name **physics-informed neural networks** and, more importantly, the machinery: modern autodiff makes the derivative of a network with respect to its input essentially free to write down, and that is the operation the whole method rests on.

It is worth registering the pattern, because Week 1 made the same point about backpropagation and Week 4 made it about convolution. The idea was available for decades. What changed was the surrounding infrastructure, and then it looked sudden.

QUIZ WATCH · WEEK 6

Likely: state the difference between a statistical regularity and a physical law as priors, and say why it matters. Explain what "the physics-informed model used zero observations" means and why it is possible. Name three of the six methods and the piece of knowledge each builds in.

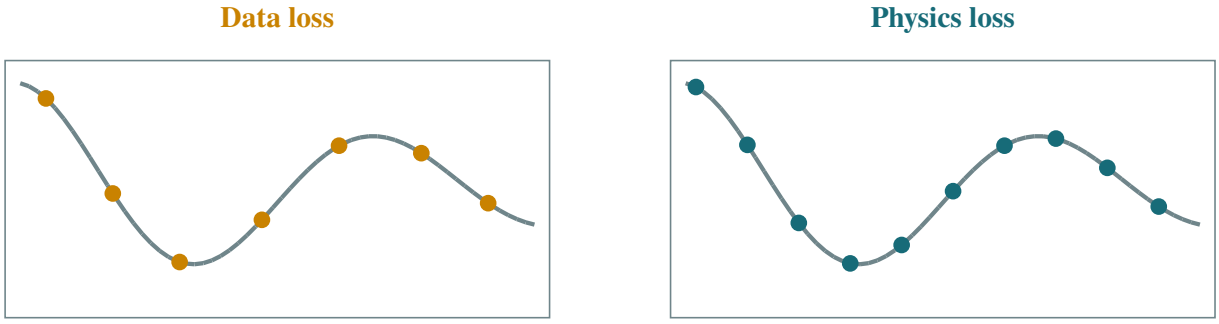
2 What "Physics-Informed" Actually Means

COVERS	The change to the loss function, and nothing else
TIME	About 16 minutes
IN BRIEF	The network is the same, the optimiser is the same, the training loop is the same. One term in the loss is replaced, and everything follows from that.

Here is the entire method, before any notation.

Ordinary supervised learning compares the model's output against measurements, at the places where you have measurements.

Physics-informed learning compares the model's output against an **equation**, at places where you have nothing at all.



The dots are the only places each loss is ever evaluated. On the right they are free.

Look at the dots in that figure. On the left, each dot is a measurement, which somebody had to obtain: a sensor, an experiment, a labelled example, a cost. On the right, each dot is a point in the input space where the equation is checked, and those cost nothing. You can put a million of them anywhere you like, because checking an equation does not require knowing the answer.

KEY IDEA

A data loss asks: **does the model agree with what we measured?**

A physics loss asks: **does the model obey the rule that the true answer must obey?**

The second question can be asked anywhere, for free, without measuring anything, including in regions where measurement is impossible. That is the source of every advantage in this week.

2.1 The loss, written out

Take a mass on a spring with friction. Newton's second law gives:

$$mu''(t) + cu'(t) + ku(t) = 0$$

where $u(t)$ is displacement, m is mass, c is the friction coefficient and k is the spring stiffness. This is the equation the true trajectory satisfies at every instant.

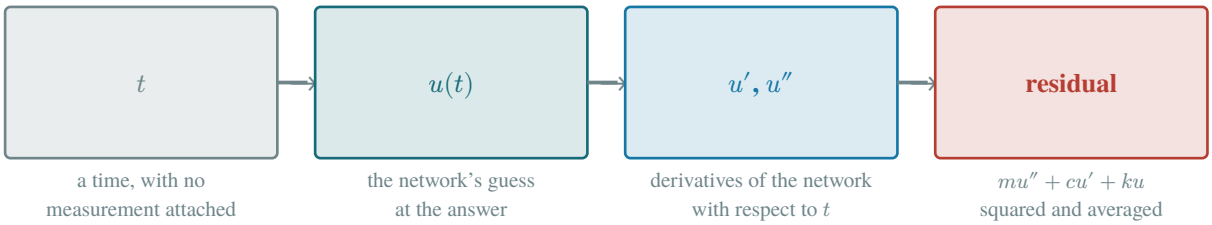
Now let a neural network be u. Feed it a time t, get a displacement out. Compute the derivatives u' and u'' **of the network with respect to its input**, and form:

$$\text{residual}(t) = mu''(t) + cu'(t) + ku(t)$$

If the network happened to be the true solution, this would be zero at every t. So make it the loss:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\left(\text{residual}(t_i)\right)^2$$

evaluated at N points t_i scattered across the interval you care about. Those points are called **collocation points**, and they are the free dots on the right of the figure.



Nowhere in this pipeline does the true solution appear. That is the whole point.

IN PLAIN ENGLISH

A useful way to hold this in mind. Suppose you are marking a student's homework, and you have lost the answer sheet.

You cannot check their answer against the truth, because you do not have it. But you **can** substitute their answer back into the original question and see whether it works. If the question was "find x such that $x^2 = 9$" and they wrote 4, you do not need the answer sheet: $16 \neq 9$ and they are wrong.

A physics loss is exactly that check, applied to a differential equation, at a few hundred places, by gradient descent. And exactly as with the homework, you never needed the answer.

2.2 What is the same, and what is different

Component	Week 3 and Week 4	Week 5
The network	a plain feedforward network	**the same**, though tanh rather than ReLU: Chapter 9
Forward pass	inputs to outputs	**the same**
The loss	distance to measured labels	**distance to zero of an equation's residual**
Backward pass	derivative of loss with respect to weights	**the same**, but the loss now contains derivatives with respect to the input: Chapter 7
Optimiser	Adam	**the same**
The loop	forward, loss, backward, step	**the same**

Only one row changed. Week 4 made the same observation about architectures and it holds here too: the training loop of Week 3 has now survived three weeks of apparently radical developments, and it will survive the rest of the module.

COMMON MISTAKE

"A PINN is a new kind of neural network."

It is not. It is an ordinary network with an unusual loss function. There is no PINN layer, no PINN architecture and no PINN activation, and if you go looking for one you will be confused for a while.

What is unusual is that the loss contains derivatives of the network with respect to its **input**, which requires a capability Weeks 3 and 4 never needed. That is Chapter 7, and it is the only genuinely new machinery in the week.

WHY THIS MATTERS

Understanding that only the loss changed tells you immediately what transfers and what does not.

Everything you know about optimisation transfers: learning rates, Adam, initialisation, the vanishing gradient. Everything you know about architectures transfers, including Week 4′s whole argument, since a physics loss can be put on top of a convolutional or attention-based model just as easily.

What does not transfer is your intuition about **what a small loss means**. In Weeks 3 and 4 a small training loss meant the model fitted the training data, and the danger was overfitting. Here a small loss means the model satisfies the equation at the collocation points you chose, and the dangers are entirely different: you may have chosen the points badly, or weighted the terms badly, or written down the wrong equation. Chapters 10, 11 and 12 take those in turn.

› OPEN THE PINN SANDBOX

Open `interactive/pinn-sandbox.html` and press **Train**. The page contains a real automatic differentiation engine and genuinely trains a physics-informed network in your browser: it is not a replay of a recorded run.

Watch the two numbers in the readout. The **physics loss** is what is being minimised. The **true RMSE** is measured against the exact solution, which the training procedure never sees. They fall together, and the fact that they fall together is the entire claim of the week made visible in about fifteen seconds.

QUIZ WATCH · WEEK 6

Very likely: state what a physics loss measures and how it differs from a data loss. Explain what a collocation point is and why they are free. State which components of the Week 3 training loop change in a PINN, and which do not.

3 Reading a Differential Equation

COVERS	What these equations say, in words, for anybody whose calculus is rusty
TIME	About 14 minutes; skip if differential equations are already comfortable
IN BRIEF	A differential equation is a sentence about how fast things change. Once you can read it as a sentence, the rest of the week is straightforward.

This week is full of expressions like $mu'' + cu' + ku = 0$. If those are unfamiliar or half-remembered, this chapter makes them readable. Nothing here is examinable on its own; it is here so the rest of the booklet works.

3.1 The three symbols

Let $u(t)$ be some quantity that changes over time: a position, a temperature, a concentration.

- $u(t)$ is **the value** at time t. Where the mass is.
- $u'(t)$ is **the rate of change**. How fast it is moving, its velocity.
- $u''(t)$ is **the rate of change of the rate of change**. How fast the velocity is changing, its acceleration.

That is all. A dash means "how fast is the thing below changing".

> **IN PLAIN ENGLISH**
>
> Driving a car gives you all three directly.
>
> u is **where you are**, the reading on a map.
>
> u' is **your speed**, the reading on the speedometer.
>
> u'' is **how hard you are accelerating or braking**, what you feel pressing you into the seat.
>
> These are genuinely different quantities. You can be far from home (u large) while stationary ($u' = 0$) and while not accelerating ($u'' = 0$). Sitting in a traffic jam in Galway is exactly that state.

3.2 Reading the oscillator equation as a sentence

$$mu'' + cu' + ku = 0$$

Take it term by term.

- mu'' is **mass times acceleration**, which by Newton's second law is the total force.
- cu' is a force proportional to **velocity**, pointing backwards. That is friction: the faster you go, the harder it pushes back.
- ku is a force proportional to **displacement**, pointing backwards. That is the spring: the further you stretch it, the harder it pulls.

So the sentence is: **the total force equals the friction force plus the spring force, and they balance to zero.**

Which is to say: $F = ma$, rearranged. The equation is not an abstraction; it is Newton, written so a computer can check it.

> ✎ **WORKED EXAMPLE: Reading the heat equation the same way**
>
> $$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

The curly ∂ means "rate of change with respect to one variable while the others are held still", because u now depends on both position x and time t. That is the only difference from a dash.

- The left side is **how fast the temperature at this point is changing**.
- The right side involves the second derivative in **space**, which measures **curvature**: how much the temperature here differs from the average of its neighbours.

So the sentence is: **a point heats up when it is colder than its surroundings, and cools when it is hotter, at a rate set by α.**

Which is what heat does. A hot spot in a cold rod has strong downward curvature and cools fast. A flat region has no curvature and does not change. Everything the heat equation predicts, including the fact that a sine profile keeps its shape and just shrinks, follows from that one sentence.

3.3 Why the conditions are separate

An equation about rates of change does not say where you started. Many different trajectories obey the same rule.

$u = 0$ satisfies the oscillator equation: a mass sitting still at the origin experiences no forces and stays there. So does an oscillation of any amplitude and any phase. The equation constrains the **shape** of the motion, not which particular motion happened.

Initial conditions pick one out: $u(0) = 1$ and $u'(0) = 0$ says "pulled one unit to the side and released from rest". Now there is exactly one answer.

KEY IDEA

Equation plus conditions determines the solution. Equation alone does not.

This is why Chapter 10 spends so long on how the conditions are imposed. Get them wrong and a PINN will happily converge to a different, perfectly valid solution of the same equation, including the trivial one.

TRY IT YOURSELF

Read each of these as an English sentence before continuing.

1. $u' = -ku$. (Answer: the rate of decrease is proportional to how much is left. Radioactive decay, or a cooling cup of tea.)
2. $u' = ku(1 - u)$. (Answer: growth proportional to both the current population and the room remaining. The logistic equation.)
3. $u'' = -g$. (Answer: acceleration is constant and downward. A thrown ball, ignoring air.)

QUIZ WATCH · WEEK 6

Not examinable directly, but you will not be able to answer anything else in the week without it. If reading these is effortful, do the three exercises above properly.

4 Where Scientific Knowledge Lives

COVERS Five kinds of prior knowledge, and which method exploits each
TIME About 15 minutes; the map of the week
IN BRIEF Governing equations are only one kind of known physics. Symmetries, conservation laws, inequality constraints and known process structure are all exploitable, by different machinery.

"Physics-informed" is usually used to mean "there is a differential equation in the loss". That is the most famous case and it is not the general one. It helps to see the whole range, because your problem may offer one kind of knowledge and not another.

Kind of knowledge	Example	How it is used	Ch.
Governing equation	Newton's law, the heat equation, Navier-Stokes	residual in the loss, or hard-constrained	8 to 13
Symmetry	energy does not change when you rotate a molecule	built into the features or the architecture	14, 15
Conservation law	energy, mass and momentum are not created	inherited from the equation, or imposed directly	12, 22
Inequality constraint	entropy never decreases; concentration is never negative	architecture that cannot output a violation	22
Known process structure	diffusion destroys structure at a known rate	used as scaffolding for a harder task	20, 21

4.1 Why the last two are underrated

The first three are the well-known cases. The last two deserve a moment because they are less obvious and often easier to exploit.

An inequality constraint is often available when an equation is not. You may not know the governing equation for a chemical process, but you certainly know a concentration cannot be negative, and a model whose final layer is a softplus **cannot** produce one. That is a hard guarantee obtained for free, and it is the same kind of guarantee as Chapter 10′s hard constraint.

Known process structure is the most interesting and the least intuitive. You may not know the distribution of natural images, and nobody does. But you know exactly what happens when you add Gaussian noise to one, because that is a process you defined. Diffusion models exploit precisely that: they use a known, unlearned physical process as the scaffolding on which a hard learning problem becomes a sequence of easy ones. Chapter 20.

> **IN PLAIN ENGLISH**
>
> Suppose you are trying to reconstruct a shredded document.

Knowing the **language** it was written in is a governing equation: a strong rule about what is allowed.

Knowing that the shredder cuts **vertical strips of exactly 6 mm** is known process structure. It does not tell you anything about the content, and it makes the problem dramatically easier, because now you know exactly what kind of damage was done and can reason backwards through it.

Diffusion models are entirely the second kind of knowledge, and that is why they belong in this week even though there is no differential equation in their loss.

◈ THE ASSUMPTION: THE EQUATION IS RIGHT

The plain sentence: the physics I have written down actually governs the system I am modelling.

Where it is true: well-characterised engineering systems, controlled experiments, anything where the mechanism is understood and the coefficients are measured.

Where it is false: biological systems with unmodelled interactions, engineering systems with unmodelled wear or contamination, anything where a coefficient was estimated badly.

What it buys when true: everything in this week. Data efficiency, extrapolation, conservation, reliability.

What it costs when false: more than you would expect. Chapter 23 measures it, and the answer is that a wrong law is worse than no law.

WHY THIS MATTERS

Knowing which kind of knowledge you have determines which method is even applicable, and this is the first question to ask on a real project.

If you have a governing equation, PINNs and neural operators are on the table. If you only have a symmetry, an equivariant architecture is the right move and a PINN is not. If you have neither but you know a quantity must stay positive or bounded, the right answer may be an architectural constraint costing three lines. And if you have a known corruption process, you may be in the diffusion family without realising it.

Reaching for a PINN because it is the famous method, when what you actually possess is a symmetry, is a common and expensive mistake.

QUIZ WATCH · WEEK 6

Likely: name three kinds of scientific prior knowledge other than a governing equation, and say how each is used. Given a described problem, identify what knowledge is available and which method suits it.

5 The Data Regime That Makes This Pay

COVERS	Where physical priors help most, where they help least, and why that is exactly Week 4′s bargain
TIME	About 12 minutes
IN BRIEF	Structure substitutes for data. Physics is the strongest structure available, so the payoff is largest where data is scarcest, which is where science lives.

Week 4, Chapter 21 stated a bargain: structure built into a model substitutes for data, so with little data the model whose assumptions match reality wins, and with enormous data the model that assumes less can learn the structure itself and then wins.

This week is the extreme end of that curve, and it is worth seeing why it does not contradict the second half of the bargain.

KEY IDEA

A physical law is not an approximation that enough data could improve upon. It is exactly true. So the second half of Week 4′s bargain does not apply: there is no amount of data at which "learn it from examples instead" produces a better answer than "it is exactly this".

What the data-rich regime **does** give you is the option of not bothering. If you have a billion examples, learning the physics implicitly may be accurate enough and less work. That is a resource argument, not a correctness one.

MEASURED IN THE LAB

The data-efficiency curve, from `pinn.py` and Chapter 8′s table, read as a graph:

Observations	Data-only error	Physics-informed error
0	not possible	**0.00135**
4	0.31289	-
8	0.06556	-
16	0.04326	-
32	0.02962	-
64	0.02648	-

The data-only column is flattening out around 0.026, which is roughly the measurement noise. More data will keep improving it slowly, and it has a long way to go to reach 0.00135.

The physics-informed row cannot be plotted on the same axis, because its x-coordinate is zero.

5.1 Where the payoff is largest

Situation	Why physics-informed methods help
Very little data	The prior does the work the data cannot. This is most engineering and most science.
Data in one region only	The equation is known everywhere, including where no instrument can go. Chapter 8′s extrapolation result.

Noisy measurements	The physics term penalises trajectories that are not physically possible, which noise is not.
Safety matters	You get guarantees about quantities you never trained on. Chapter 12.
Inverse problems	You have the equation and want an unknown coefficient. Chapter 24.

And where it does not:

Situation	Why not
No reliable equation	Most of biology, economics, and human behaviour. A guessed equation is worse than none: Chapter 23.
Abundant data and a fast classical solver	If a finite-element solver already does this in a second, a PINN is a worse tool. Chapter 23.
Very high dimension with fine structure	Collocation points must fill the domain, and that scales badly. Chapter 13.

WHY THIS MATTERS

The honest framing to carry into a job interview or a project meeting: **physics-informed machine learning is not a better way to solve differential equations.** For most standard equations on standard domains, sixty years of numerical analysis has produced solvers that are faster and come with error bounds, and a PINN will lose to them.

It is a better way to do several other things: to combine sparse messy data with known physics, to solve problems where the geometry or the coefficients are unknown, to handle inverse problems, and to build fast surrogates for a family of problems, which is Chapter 18.

Saying this clearly is a mark of understanding the field rather than its marketing, and Chapter 23 gives the evidence.

QUIZ WATCH · WEEK 6

Likely: state where physical priors help most and give two situations where they do not. Explain why Week 4's "enough data beats structure" argument does not apply in the same way to exactly-true laws.

6 Reading the Notation of This Week

COVERS A decoder for the symbols in this booklet and in the papers you will meet
TIME About 8 minutes; return here as needed
IN BRIEF Small chapter, disproportionate payoff, exactly like Week 4's Chapter 5.

You will see	It means	In plain English
u', $\dot{u}$, $\frac{du}{dt}$	first derivative	how fast u is changing
u'', $\ddot{u}$, $\frac{d^2u}{dt^2}$	second derivative	how fast the rate of change is changing
$\frac{\partial u}{\partial t}$	partial derivative	rate of change with respect to t, holding other inputs still
∇u	gradient	the vector of all first derivatives; the direction of steepest increase
$\nabla^2 u$, Δu	Laplacian	sum of the second derivatives; how much u differs from its neighbours' average
$\mathcal{N}[u] = 0$	a differential operator applied to u	a compact way of writing "whatever equation we are enforcing"
collocation points	where the residual is evaluated	locations you chose, with no data attached
λ or w	a loss weight	how much one term of the loss counts against another; Chapter 10
residual	what the equation gives when your candidate is substituted in	how wrong the answer is, in the equation's own units
$G : a \mapsto u$	an operator	a map from a function to a function, not a number to a number; Chapter 18
equivariant	transform the input, the output transforms the same way	Week 4, Chapter 9, same idea
invariant	transform the input, the output does not change	Week 4, Chapter 9, same idea
β_t, α_t, $\overline{\alpha}_t$	a diffusion noise schedule	how much noise is added at each step, and how much signal survives; Chapter 20
score	$\nabla \log p(x)$	which direction makes the data more likely; Chapter 21

IN PLAIN ENGLISH

Two pieces of vocabulary cause most of the confusion, and both are simpler than they sound.

An operator is a machine that takes a whole function and gives back a whole function. "Differentiate" is an operator: feed it sin, get back cos. So is "solve this PDE with the following initial condition": feed it a starting temperature profile, get back the entire future evolution. Chapter 18 learns one of those with a network.

The score is a direction, not a number. At any point in space it points the way towards where the data is denser. If you keep stepping in that direction with a bit of randomness, you end up in a typical place for the data, which is exactly what generating a sample means.

QUIZ WATCH · WEEK 6

Not examinable directly. Reliable comprehension of the symbols is assumed by everything else.

7 Differentiating a Derivative

COVERS	The one genuinely new piece of machinery, and why Week 3′s hand-written backprop stops being enough
TIME	About 20 minutes; the technical heart of the week
IN BRIEF	A PINN's loss contains the network's second derivative. Training it needs the derivative of that, which means an autodiff engine whose gradients are themselves differentiable.

7.1 The problem

Week 3 wrote backpropagation by hand. Week 4 wrote it again for convolutions, recurrence and attention. In every case the pattern was the same: differentiate the **loss** with respect to the **weights**.

A PINN needs something new. Its loss is

$$\mathcal{L} = \text{mean}\Big((mu'' + cu' + ku)^2 \Big)$$

where u' and u'' are derivatives of the network with respect to its **input**. To train, you need $\partial \mathcal{L} / \partial$ weights, which means differentiating an expression that already contains derivatives.

So you need the derivative **of** a derivative, with respect to a different variable. Doing that by hand for a three-layer network is possible and thoroughly unpleasant, and it has to be redone for every change of architecture. The lab builds the machinery instead.

KEY IDEA

In an ordinary autodiff engine, each backward pass returns **numbers**. Once you have the gradient, it is data, and data cannot be differentiated.

The trick is to make every backward pass return **nodes in the computation graph**, built from the same operations as the forward pass. Then the gradient is itself an expression, so you can differentiate it again, as many times as you like.

That single design decision is the whole of `autodiff.py`, and it is what PyTorch means by `create_graph=True`.

✎ WORKED EXAMPLE: The engine, in the only three lines that matter

From `autodiff.py`. Here is the backward rule for multiplication:

```
def mul(a, b):
    def vjp(g):
        return (sum_to(mul(g, b), a.value.shape),
                sum_to(mul(g, a), b.value.shape))
    return Node(a.value * b.value, (a, b), vjp, "mul")
```

Note what `vjp` returns. Not `g.value * b.value`, which would be an array. It returns `mul(g, b)`, which is **another node**, constructed with the very function being defined.

And here is the accumulation inside `grad`:

```
grads[id(parent)] = add(grads[id(parent)], contribution)
```

That `add` is a graph operation, not a numpy one. So the gradient that comes out of `grad` is a live expression with parents and backward rules of its own, and calling `grad` on it works exactly as it did the first time.

Everything else in the file is bookkeeping: broadcasting, shapes, and about a dozen operations each written in the same style.

7.2 Why not just use finite differences

There is an obvious alternative. Approximate the derivative numerically:

$$u'(t) \approx \frac{u(t+\varepsilon) - u(t-\varepsilon)}{2\varepsilon}$$

This requires no machinery at all. It is also a bad idea, for reasons worth understanding, because they explain why PINNs became practical only when autodiff did.

MEASURED IN THE LAB

The lab measures the gap directly, in `check_network_derivative()`:

Quantity	Autodiff	Central difference	Relative gap
first derivative	+0.38072959	+0.38072959	1.4e-11
second derivative	−0.03664877	−0.03664985	3.0e-05

The disagreement is the finite difference's error, not the engine's. The engine differentiates symbolically, so its answer is exact to machine precision; the difference quotient is an approximation whose error you can see growing from the first derivative to the second.

The problems compound quickly:

1. **Accuracy degrades with each order.** A second derivative by finite differences divides by ε^2, so rounding error is amplified enormously. A fourth-order PDE would be hopeless.
2. **Cost scales with dimension.** Each derivative needs extra forward passes, per input dimension. In three spatial dimensions plus time, that is a great deal of extra work at every collocation point.
3. **You must choose ε.** Too large and the approximation is poor; too small and rounding dominates. The lab's own gradient checks show that sweet spot moving between orders.

MEASURED IN THE LAB

The engine is verified against exact calculus rather than against another approximation. From `check_engine()`, comparing to hand-derived derivatives of sin, tanh and $e^x x^3$ up to third order:

worst relative error: 1.22e-16

which is the rounding error of 64-bit floating point. The derivatives are not approximately right; they are the same computation performed twice.

⌕ DEEPER DETAIL: A numerical trap that only appears at the third derivative

Worth recording because it cost real time to find, and because it is exactly the kind of bug that does not announce itself.

The lab originally implemented squaring as `power(a, 2)`. That differentiates to `2 * power(a, 1)`, which differentiates to `2 * power(a, 0)`, and the **third** derivative reaches `power(a, -1)`, which is a division by zero wherever $a = 0$.

A PINN evaluates its residual at $t = 0$, because that is where the initial condition lives. So the bug fires precisely at the most important point in the domain.

The fix is to write squaring as `mul(a, a)`, which keeps every derivative polynomial and finite. The lesson generalises: an operation that is perfectly safe when you differentiate it once may not be when you differentiate it three times, and this week differentiates everything at least twice.

✓ IN PRACTICE: In PyTorch, this is one keyword

You will not write an autodiff engine in practice. You will write:

```
u = net(t)
u_t = torch.autograd.grad(u.sum(), t, create_graph=True)[0]
u_tt = torch.autograd.grad(u_t.sum(), t, create_graph=True)[0]
residual = m * u_tt + c * u_t + k * u
loss = (residual ** 2).mean()
```

`create_graph=True` is the whole of this chapter. It tells autograd to build the gradient as a differentiable expression rather than as data. Leave it out and the second line works, the third fails, and the error message will not obviously be about this.

Also note `t` must have `requires_grad=True`, because you are differentiating with respect to an **input**, which is not something ordinary training ever does.

QUIZ WATCH · WEEK 6

Very likely: explain why a PINN's loss requires differentiating a derivative. State what makes an autodiff engine capable of higher-order derivatives. Give two reasons finite differences are unsuitable. In PyTorch, name the argument that makes this work.

8 The PINN, End to End

COVERS	The whole method assembled, with the measured results
TIME	About 20 minutes; the chapter the lab implements
IN BRIEF	Network, hard constraint, residual, loss, optimiser. Then the four results that make the case.

8.1 The complete recipe

1. **Choose a network.** Input: the coordinates (t, or x and t). Output: the quantity. Smooth activation, because you will differentiate it twice. Chapter 9.
2. **Scale the inputs** to roughly $[-1, 1]$. Chapter 9 explains why this matters more here than usual.
3. **Impose the conditions** by construction if you can. Chapter 10.
4. **Choose collocation points** across the domain. Chapter 11.
5. **Compute the residual** using autodiff for the input derivatives. Chapter 7.
6. **Minimise the mean squared residual**, plus a data term if you have measurements.
7. **Check what you got**, because a small loss is not evidence. Chapters 11 and 23.

That is the lab's `Solver` class in its entirety, and it is about forty lines.

8.2 The four results

MEASURED IN THE LAB

One: it works with no data at all.

Model	Observations	Error
physics-informed	**0**	**0.00135**
data-only	64	0.02648

Same network, same optimiser, same epochs. The physics-informed model is twenty times more accurate having seen nothing.

MEASURED IN THE LAB

Two: it does not fall apart outside the data. Both models get measurements only on the first 40% of the interval. Both are scored on the rest.

Model	Where data existed	Beyond the data	Degradation
data-only	0.0181	0.4998	**27.6 times worse**
physics + the same data	0.0086	0.0087	**1.0**

The physics-informed model's error is **the same** outside the data as inside it. Not similar: the same to two decimal places.

This is the reliability argument, and reliability is usually worth more than accuracy. A model that is excellent inside its training range and arbitrary outside it is dangerous precisely because the failure is silent: nothing in its output announces that it has left the region where it knows anything.

MEASURED IN THE LAB

Three: noise hurts it less. Same number of measurements, increasing measurement error:

Noise	Data-only	Physics + data	Ratio
0.00	0.0069	0.0014	5.0
0.10	0.0921	0.0454	2.0
0.30	0.2658	0.1363	2.0

Roughly twice as accurate at every level. The mechanism is worth naming: the physics term is a **regulariser**, but not the kind from Week 3, Chapter 10. Weight decay penalises complexity. A physics residual penalises **impossibility**. Noise is not a physically possible trajectory for a damped oscillator, so the physics term suppresses it, and nobody had to choose a smoothing parameter.

MEASURED IN THE LAB

Four: it works on a PDE too, and better. From `heat.py`:

Model	Observations	Error
physics-informed	**0**	**0.00022**
data-only	128	0.01741

Seventy-nine times more accurate, using no data.

EVERYDAY ANALOGY

Why the extrapolation result is the one to remember.

Suppose you are asked where a thrown ball will land, having watched only the first half-second of its flight.

A model fitted to those observations can describe that half-second beautifully and has no principled opinion about the rest. Extended, it will do something smooth and arbitrary, and it will not warn you.

A model that knows **the ball is subject to gravity** can tell you where it lands, because the equation applies for the whole flight, not just the observed part. It is not extrapolating from data; it is integrating a law.

That is the difference between a curve fitted to a trajectory and a model of the motion, and it is the whole of Chapter 16 as well.

▸ OPEN THE PINN SANDBOX

Tab 2, “Physics against data”. Drag the split slider to the left so the data covers less of the interval, and press **Train both**. Two models, same architecture, same measurements.

Watch the four numbers underneath. The two “where data was” numbers are similar. The two “beyond it” numbers are not, and the gap widens as you shrink the data region. The page trains both models live, so the result is genuinely reproduced each time rather than replayed.

QUIZ WATCH · WEEK 6

Very likely: state the recipe in order. Reproduce the extrapolation result and explain why the degradation factor is 1.0 for one model and 27.6 for the other. Explain in what sense a physics loss is a regulariser and how it differs from weight decay.

9 Choosing the Network

COVERS The architectural choices a physics loss changes, and why ReLU is now wrong
TIME About 14 minutes
IN BRIEF Week 4 said ReLU everywhere. A PINN cannot use it, and the reason is a one-line argument worth knowing.

9.1 Why ReLU is disqualified

A PINN's loss contains u'', the second derivative of the network with respect to its input.

ReLU is piecewise linear. Its first derivative is a step function, and its **second derivative is zero almost everywhere**. A network built from ReLUs therefore reports that its own curvature is zero at essentially every point.

Feed that into the oscillator residual and the acceleration term vanishes. The loss will be minimised by something that has nothing to do with the equation you meant to enforce.

KEY IDEA

If your loss contains an n-th derivative, your activation function must be differentiable at least n times and not trivially so.

This is the first example of something that recurs all week: **a physics constraint reaches back and changes an architectural choice you thought was settled.** Week 4's default was ReLU, chosen for cheap gradients and no saturation. Week 5 cannot have it, and this is not negotiable.

Activation	Second derivative	Verdict for a PINN
ReLU	zero almost everywhere	**unusable** for second-order equations
Leaky ReLU	zero almost everywhere	**unusable**, same reason
tanh	smooth, non-trivial	**the standard choice**
sine	smooth, non-trivial	excellent for oscillatory solutions (the SIREN idea)
GELU, SiLU, softplus	smooth	usable; less common in this literature

MEASURED IN THE LAB

The lab tried both smooth options on the oscillator. With the same budget, `tanh` reached about 1.4e-03 and `sin` about 4.7e-04 on an early sweep.

This is not surprising and it is worth understanding rather than memorising: the true solution is a decaying sinusoid, so sinusoidal activations can represent it with fewer units. That is Week 4's thesis again, one level down. **The activation function is also an assumption about the shape of the answer.**

9.2 Input scaling, which matters more here

Feature scaling was ordinary good practice in Week 3. Here it is close to mandatory, and the reason is specific.

If the input runs from 0 to 10, and you feed that directly to a tanh, the network saturates immediately and the useful gradient range covers a tiny fraction of the domain. Worse, the **second** derivative's magnitude scales as the square of the input scale, so the residual's terms end up on wildly different scales and the loss is dominated by whichever happens to be largest.

The lab maps t into $[-1, 1]$ before the network sees it:

```
raw = self.net(add(mul(t, 2.0 / T), -1.0))
```

One line, and without it the training is markedly worse.

9.3 Depth, width and the hard part

For the problems in this lab, three or four hidden layers of 32 to 64 units is ample. The lab uses `[1, 32, 32, 32, 1]`, which is 2,209 parameters. Larger networks do not obviously help and train more slowly.

The genuinely difficult part is not capacity. It is **optimisation**, and this is where PINNs earn their reputation for being finicky.

⌕ DEEPER DETAIL: Why PINN losses are hard to optimise

Three reasons, all worth knowing because they explain most of the practical advice you will read.

The loss is a sum of terms with different characters. A residual term and a boundary term have different units and different curvature, so the loss surface is badly conditioned in a way an ordinary supervised loss is not. Chapter 10 measured the consequence.

Differentiating twice sharpens everything. The residual depends on the network's curvature, and curvature is far more sensitive to a weight change than the value is. Small parameter moves cause large residual moves.

There is a trivial minimum nearby. For a homogeneous equation, $u = 0$ satisfies it exactly. The optimiser can find that instead, and the only thing preventing it is the conditions. Hard constraints remove this failure entirely.

Two practical consequences. First, Adam is essentially required; plain gradient descent struggles badly here. Second, a common trick in the literature is to finish with L-BFGS, a second-order optimiser, after Adam has got close. That is unusual in deep learning generally and standard in this corner of it, precisely because the problems are small and badly conditioned rather than large and noisy.

COMMON MISTAKE

"My PINN is not converging, so I will make the network bigger."

Almost always the wrong move. Check, in this order: is your activation smooth; are your inputs scaled; are your conditions imposed as hard constraints; are your loss terms balanced sensibly; do you have enough collocation points. Every one of those is more likely to be the problem than capacity, and every one is cheaper to fix.

QUIZ WATCH · WEEK 6

Very likely: explain why ReLU cannot be used in a PINN with a second-order equation. Name two suitable activations. Explain why input scaling matters more for a PINN than for an ordinary regressor. Give two reasons PINN losses are hard to optimise.

10 Hard Constraints, Soft Penalties, and a 400-Fold Difference

COVERS	How to impose initial and boundary conditions, and why the obvious way is unreliable
TIME	About 18 minutes; the most practically useful chapter in the week
IN BRIEF	Adding a penalty term introduces a weight nobody can derive. Building the condition into the functional form removes the question entirely.

A differential equation alone does not determine a solution. The oscillator equation is satisfied by every amplitude and every phase, and by $u = 0$. The initial conditions $u(0) = 1$ and $u'(0) = 0$ are what pick out one answer.

So a PINN needs to impose them, and there are two ways.

10.1 The soft way, and its problem

Add a penalty:

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{physics}}}_{\text{satisfy the equation}} + \lambda \underbrace{\Big((u(0) - 1)^2 + (u'(0))^2 \Big)}_{\text{satisfy the conditions}}$$

This is easy to write and it is what most tutorials do. It also introduces λ, a number that is not determined by the problem, not derivable from theory, and consequential.

MEASURED IN THE LAB

From `experiment_constraint_style()`. Identical network, identical everything, only the value of λ changing:

How the condition is imposed	$u(0)$, should be 1	$u'(0)$, should be 0	Test error
soft penalty, $\lambda = 1$	0.99988	−0.00005	0.00154
soft penalty, $\lambda = 50$	0.99941	−0.00019	**0.32225**
soft penalty, $\lambda = 500$	0.99953	−0.00048	**0.66444**
hard constraint	**1.00000**	**0.00000**	**0.00135**

A spread of **over 400 times** in accuracy, caused entirely by a number nobody can compute in advance.

Now look at the middle two columns, because the failure is not the one most people would guess. **Every version satisfies the initial condition to four decimal places.** The bad runs are not violating the condition. They are satisfying it while neglecting the equation, because their loss told them the condition mattered five hundred times more than the physics.

IN PLAIN ENGLISH

Think of a builder given two instructions: **make the house structurally sound** and **paint the door red**.

If you tell them the red door is five hundred times more important, you will get a beautifully painted door on a house that is not quite standing up. They followed the instructions. The instructions were badly weighted.

A soft penalty is exactly this. The optimiser is not being obtuse; it is optimising the thing you actually wrote down.

10.2 The hard way

Instead of asking the network to satisfy the conditions, construct a function that cannot fail to:

$$u(t) = u_0 + v_0 t + s(t)N(t), \qquad s(t) = \frac{t^2}{1+t^2}$$

where N is the neural network. Check it:

- At $t = 0$: $s(0) = 0$, so $u(0) = u_0$. Exactly, for any weights.
- Differentiating: $s'(0) = 0$ as well, so $u'(0) = v_0$. Exactly, for any weights.

The conditions now hold **identically**, at initialisation, at every epoch, regardless of what the optimiser does. The loss has exactly one term, there is nothing to balance, and there is no λ.

Soft penalty

loss = residual + λ × (violation)²

The condition is a **wish**. The optimiser may trade it away for a smaller residual, and λ is a number nobody can derive. Measured spread in this lab: **over 400 times**.

Hard constraint

$u(t) = u_0 + v_0 t + s(t)N(t)$, with $s(0) = s'(0) = 0$

The condition is **arithmetic**. It holds for any weights, from the first epoch, and the loss has only one term. Measured: $u(0) = 1.00000$ exactly.

⚖️ THE LAW: INITIAL CONDITIONS

The plain sentence: the system started at a known place with a known velocity.

How it is built in: not as a penalty but as the shape of the function itself. The network's output is wrapped in an expression that satisfies the conditions automatically.

What it buys: a single-term loss, no weight to tune, exact satisfaction from the first epoch, and in this lab a 400-fold reduction in the spread of achievable accuracy.

What it costs: you have to construct the wrapper, which requires thinking about the geometry of your domain. For simple domains this is easy; for complicated ones it can be genuinely hard, and that is the honest limitation.

✎ WORKED EXAMPLE: The same trick on a PDE, where it pays even better

The heat equation has three condition families: an initial temperature profile, and two boundaries held at zero. As penalties that is three more terms and at least three more weights.

The lab's `HeatSolver` uses:

$$u(x,t) = \sin(\pi x)[1 + tN(x,t)]$$

Check all three:

- At $t = 0$ the bracket is 1, so $u = \sin(\pi x)$. The initial condition holds exactly.
- At $x = 0$, $\sin(0) = 0$, so $u = 0$. The left boundary holds exactly.
- At $x = 1$, $\sin(\pi) = 0$, so $u = 0$. The right boundary holds exactly.

The loss is again a single term. And the result is the best in the lab:

Model	Observations	Error
physics-informed	**0**	**0.00022**
data-only	8	0.17098

data-only	32	0.04507
data-only	128	0.01741

79 times more accurate than a data-only model given 128 measurements, using none.

⌕ DEEPER DETAIL: Why loss balancing is an open problem, not just a chore

It would be reasonable to assume somebody has solved the weighting question. They have not, and knowing that is useful.

The difficulty is that the terms have different units, different scales and different curvature, and their relative importance changes **during** training. A weight that is right at epoch 100 may be wrong at epoch 3,000.

Several families of fix exist and are worth knowing by name. **Gradient-based balancing** rescales each term so their gradient magnitudes match. **Neural-tangent-kernel approaches** set weights from the eigenvalues of the training dynamics. **Self-adaptive weights** make λ itself a trainable parameter, maximised rather than minimised, so the optimiser is pushed towards the terms it is doing worst on.

All of them help and none is universal. This is why hard constraints, when your domain permits them, are worth real effort: they do not solve the balancing problem, they delete it.

▸ OPEN THE PINN SANDBOX

Tab 1. Untick **impose it exactly** and retrain, then tick it and retrain again. With the hard constraint the readout shows $u(0)$ exactly 1 and the solution converging cleanly; without it, you will usually see a slower and less reliable run.

Worth noticing: $u = 0$ satisfies the differential equation perfectly. With a soft penalty, the only thing standing between the optimiser and that trivial answer is the penalty term, which is why the whole business is more delicate than it looks.

QUIZ WATCH · WEEK 6

Very likely: state the two ways to impose an initial condition and give one advantage of each. Explain the measured 400-fold spread and why the u(0) column shows the failure is not what it seems. Verify that a given hard-constraint ansatz satisfies stated conditions, which is a very natural applied question.

11 Collocation Points, and the Loss You Are Really Minimising

COVERS Where the equation gets checked, how many points you need, and the failure this creates
TIME About 16 minutes
IN BRIEF Collocation points are free, which is the method's great advantage and its most dangerous property.

11.1 They cost nothing, and that is the point

A supervised model can only be trained where you have labels. Labels come from experiments, sensors, annotators and money, and there are never enough.

Collocation points are just **locations**. To evaluate $mu'' + cu' + ku$ at $t = 4.7$ you need the network's output and derivatives at 4.7, and nothing else. No measurement of the true $u(4.7)$ is involved, and none is possible for a point that was never measured.

KEY IDEA

This is what lets a PINN work in places where data is impossible to obtain: inside a reactor, at the centre of a star, at a future time, at a design that has not been built. The equation is known there, so the loss can be evaluated there.

It is also, precisely and unavoidably, the source of the method's characteristic failure. Chapter 11.

11.2 How many, and where

The honest answer is that this is a design decision with real consequences and no universal rule. Some practical guidance:

- **Enough to resolve the solution.** If the true answer oscillates, you need points dense enough to see the oscillation. The lab uses 200 points over about 1.6 periods, which is roughly 125 per period, and that is generous.
- **Denser where the solution is difficult.** Sharp fronts, boundary layers and shocks need more points than smooth regions. **Adaptive collocation** moves points towards high residual during training and is an active research area.
- **Resampled, often.** Drawing fresh random points each epoch rather than using a fixed grid acts as a regulariser and reduces the risk described below.

MEASURED IN THE LAB

What happens when there are too few is worth seeing rather than being told. In the PINN Sandbox, drag the collocation slider down to about ten points and retrain.

The physics loss still falls to a small value, because the equation genuinely **is** satisfied at those ten places. The curve between them wanders freely, and the true error is large.

The loss is not lying. It is answering a different question from the one you meant to ask. It reports agreement at the points you chose, and says nothing whatsoever about the points you did not.

COMMON MISTAKE

"The physics loss is small, so the solution is right."

This is the single most common error in applied PINN work and it is worth taking seriously, because the reassurance is so plausible.

A small physics loss means: **at the finitely many places I checked, the equation is approximately satisfied.** It does not mean the equation is satisfied elsewhere, that the solution is unique, that the right solution was found among several, or that the equation was the right one.

Three checks that cost almost nothing and are worth doing every time:

1. **Resample.** Evaluate the residual at a fresh set of points that were never trained on. If it is much larger there, you have overfitted the collocation set.
2. **Refine.** Double the number of points, retrain, and see whether the answer moves. A converged solution barely moves.
3. **Check a consequence.** If the equation implies conservation of something, measure that thing. Chapter 12.

› OPEN THE PINN SANDBOX

Tab 1. Set collocation points to 10 and press **Train**, then set them to 120 and press **Reset** and **Train** again. Compare the **physics loss** and **true RMSE** readouts in each case.

The lesson lands when you notice that the small-point run has a **lower** physics loss and a **higher** true error. That combination is the whole warning of this chapter in two numbers.

QUIZ WATCH · WEEK 6

Likely: explain what a collocation point is and why it costs nothing. Give the failure mode created by too few, and state two checks that would detect it. Explain why a small physics loss is not evidence of a correct solution.

12 What You Get For Free

COVERS	Consequences of the equation that nobody put in the loss
TIME	About 14 minutes
IN BRIEF	Energy was in no loss function and was conserved anyway. That is the reliability argument in its purest form.

Everything so far has compared models on the quantity they were trained to produce. This chapter is about the quantities nobody mentioned.

12.1 The experiment

Remove the friction from the oscillator, so $c = 0$. An undamped spring conserves energy exactly:

$$E = \frac{1}{2}mv^2 + \frac{1}{2}ku^2 = \text{constant}$$

Nobody tells either model about this. It is not in the loss, not in the architecture, not in the data. Energy is a **consequence** of the equation, derivable from it in three lines of calculus.

MEASURED IN THE LAB

From `experiment_conservation()`. The true energy is constant at 0.50.

Model	Energy at $t = 0$	Energy at $t = T$	Worst drift
data-only	0.52	0.4189	**16.3%**
physics + noisy data	0.50	0.4865	5.1%
physics only, no data	**0.50**	**0.4995**	**0.2%**

The data-only model loses a sixth of the system's energy over the interval, which is physically impossible and which nothing in its training would have flagged.

The pure physics-informed model conserves it to two parts in a thousand, having never been told that energy exists.

KEY IDEA

A model that satisfies the governing equation inherits **everything the equation implies**, whether or not you thought to check it.

A model that fits data points inherits only agreement with those points, and any consequence you care about is a matter of luck.

This is the deepest practical reason to prefer physics-informed models where they apply. Your test set covers the quantities you thought to measure. The physics covers the ones you did not.

IN PLAIN ENGLISH

The difference between a student who has memorised worked examples and one who understands the subject.

Both can answer the questions they have seen. Ask something slightly different and the first produces something plausible and wrong, while the second reasons from principles and gets it right.

You find out which you have by asking a question that was not on the practice sheet. Energy drift is that question, and the data-only model fails it while its own loss looks perfectly healthy.

12.2 A note on the middle row

The hybrid model, given noisy measurements as well as the equation, drifts more than the pure physics model. That is not noise in the experiment; it is a real and instructive effect.

MEASURED IN THE LAB

The same pattern appears in Chapter 8′s data-efficiency table:

Model	Error
physics only, no data	**0.00135**
physics + 8 noisy measurements	0.02665

Adding data made it **twenty times worse**.

The explanation is straightforward once stated. When the physics is exactly right, the equation plus the initial conditions determine the answer completely. There is nothing left for data to contribute, and the data is noisy, so it can only pull the solution away from the truth.

The lesson generalises to every hybrid model you will build: **the weight on the data term should reflect how much you trust the data relative to the model.** If your physics is exact and your sensors are poor, weight the data lightly. If your physics is approximate and your sensors are excellent, do the reverse. Setting both weights to 1 because it is the default is a decision, and usually not the right one.

✓ IN PRACTICE: Conservation as a validation tool

This gives you a test that costs almost nothing and catches real failures.

After training any dynamical model, compute a quantity the system is supposed to conserve, and plot it. Energy, mass, momentum, charge, total probability, whatever your system has. It is a few lines of code and it does not need any held-out data.

A model that drifts is telling you something is wrong even when the fit looks good, and it will often catch problems before your test set does, because your test set is drawn from the same distribution as your training set and the physics is not.

QUIZ WATCH · WEEK 6

Very likely: explain why a physics-informed model conserves energy without energy appearing in its loss. Reproduce the drift numbers and their interpretation. Explain why adding noisy data to an exactly-correct physics model made it worse, and what that implies for weighting.

13 From an ODE to a PDE

COVERS	The heat equation, what changes with two inputs, and the honest scaling limitation
TIME	About 16 minutes
IN BRIEF	Three things change: more input variables, more condition families, and collocation points that must fill an area. The third is where PINNs meet their limit.

13.1 The problem

Heat flowing along an insulated rod:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}, \quad u(0,t) = u(1,t) = 0, \quad u(x,0) = \sin(\pi x)$$

with the exact solution $u(x,t) = e^{-\alpha \pi^2 t} \sin(\pi x)$: the bump keeps its shape and shrinks exponentially.

13.2 The three changes

One: two inputs, so derivatives in two directions. The network takes (x, t) and you need $\partial u/\partial t$ and $\partial^2 u/\partial x^2$ separately. The lab keeps x and t as separate nodes joined by `concat`, so `grad` can target one without the other. This is why the engine has a concatenation operation at all.

Two: three condition families instead of two numbers. An initial condition along the whole bottom edge, and boundary conditions along both sides. As soft penalties that is three more terms and at least three more weights to balance, which makes Chapter 10′s argument stronger rather than weaker.

The lab's hard constraint handles all three at once:

$$u(x,t) = \sin(\pi x)[1 + tN(x,t)]$$

At $t = 0$ the bracket is 1, giving $\sin(\pi x)$. At $x = 0$ and $x = 1$ the sine is zero. All three conditions hold exactly for any weights, and the loss again has one term.

Three: collocation points now fill an area. A 32 by 32 grid is 1,024 points instead of 200, and this is the beginning of a real problem.

MEASURED IN THE LAB

The result, from `experiment_heat()`:

Model	Observations	Error
physics-informed	**0**	**0.00022**
data-only	8	0.17098
data-only	32	0.04507
data-only	128	0.01741

The best result in the lab: 79 times more accurate than a data-only model given 128 measurements. The hard constraint is doing a great deal of the work here, since it removes three penalty weights that would otherwise all need tuning.

⌕ DEEPER DETAIL: The curse of dimensionality, and the honest limitation

Collocation points must cover the domain, and the number needed grows exponentially with dimension.

Problem	Dimensions	Points at 32 per dimension
this lab's oscillator	1 (time)	32
this lab's heat equation	2 (space and time)	1,024
2-D flow over time	3	32,768
3-D flow over time	4	1,048,576
a 10-dimensional problem	10	about 10^{15}

So a PINN on a realistic three-dimensional time-dependent problem needs a million collocation points, each requiring several derivative evaluations, every epoch. This is why demonstrations are usually one- and two-dimensional, and why claims about PINNs replacing computational fluid dynamics should be read carefully.

Two things partly redeem this. Random sampling of collocation points scales better than a grid, because you sample where the solution is difficult rather than uniformly. And in **high** dimensions, where classical grid-based solvers fail completely, a PINN's mesh-free nature becomes an advantage rather than a handicap: there are problems in finance and in quantum chemistry with dozens of dimensions where no grid method is possible at all and a sampled approach is the only option.

So the honest summary is not "PINNs scale badly". It is: **classical solvers win in low dimensions, PINNs become interesting where meshes become impossible, and the awkward middle is genuinely contested.**

✓ IN PRACTICE: When to reach for a classical solver instead

If your problem is a standard equation, on a standard domain, with known coefficients, in one to three dimensions, **use a finite-element or finite-difference solver**. It will be faster, more accurate, and it comes with convergence guarantees that a PINN does not have.

PINNs earn their place when at least one of these holds:

1. the coefficients or the source terms are **unknown** and must be inferred from data (Chapter 24);
2. you have **sparse measurements** to combine with partial physics;
3. the geometry is awkward enough that meshing is the expensive part;
4. you need a **differentiable** solver to embed in a larger optimisation;
5. the dimension is high enough that meshes are impossible.

Being able to say this clearly is worth more in an interview than being able to implement a PINN.

QUIZ WATCH · WEEK 6

Likely: name the three things that change when moving from an ODE to a PDE. Verify that a given hard-constraint form satisfies stated boundary and initial conditions. Explain the collocation scaling problem and give one situation where it is not fatal.

14 Symmetry: The Prior That Is Exactly True

COVERS	The cleanest case in the week, and a direct continuation of Week 4, Chapter 9
TIME	About 18 minutes
IN BRIEF	A symmetry is not approximately true. Build it in and you get an exact guarantee, measured here as an error of 8.88e-16.

Week 4 showed that a convolution is **equivariant** to translation, and measured the equivariance error as exactly zero. That was a symmetry chosen for convenience: images happen to be translation-friendly.

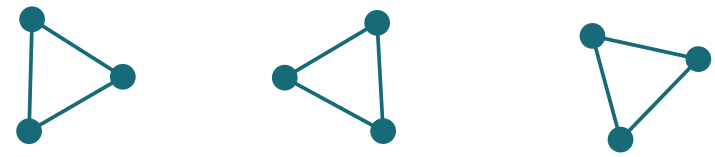
This chapter uses symmetries that come from physics, where they are not a convenient approximation but a fact.

14.1 The task

Three particles in a plane. Their potential energy is a sum over pairs of a function of the **distance** between them. The input is six numbers, the raw coordinates; the target is the total energy.

Because energy depends only on distances, it is unchanged by:

- **translation**: move all three together;
- **rotation**: turn the whole arrangement;
- **permutation**: relabel which particle is which.



as measured rotated 63° rotated 137°

All three have the same energy.

Raw coordinates: all six change.

Pairwise distances: **identical to 16 digits.**

A model that reads distances cannot get this wrong. A model that reads coordinates must learn not to.

⚖ THE LAW: ROTATIONAL INVARIANCE OF ENERGY

The plain sentence: turning a molecule round does not change how much energy it has.

Why it is exactly true: space has no preferred direction. This is not a statistical regularity about molecules; it is a symmetry of physical law itself, and by Noether's theorem it is the same fact as conservation of angular momentum.

How it is built in: compute the pairwise distances first and feed those to the network. A distance cannot change under rotation, so the output cannot either.

What it buys: exact invariance, better data efficiency, and generalisation to orientations never seen.

What it costs: if the target is **not** actually invariant, the model has discarded the information it needs and cannot recover it at any dataset size. Chapter 15.

14.2 The two models

Coordinate model: reads the six raw coordinates. It must **learn** that rotating the input does not change the answer, from examples.

Invariant model: reads the three pairwise distances, sorted. Sorting adds permutation invariance on top. Same network size otherwise.

MEASURED IN THE LAB

Is the invariance exact? Rotate the input and see whether the prediction moves.

Model	Parameters	Change under rotation	As % of output scale
coordinate MLP	2,737	1.76e-01	6.4%
invariant model	2,593	**8.88e-16**	**0.0%**

That second figure is the rounding error of 64-bit arithmetic. It is not a small error; it is the same number computed twice.

Compare Week 4′s equivariance test on convolution, which returned 0.00e+00 for the same reason. Same argument, different symmetry group.

MEASURED IN THE LAB

What does it buy in data?

Training arrangements	Coordinate MLP	Invariant model	Ratio
64	0.1533	0.0393	3.9
256	0.0841	0.0371	2.3
1,024	0.0788	0.0353	2.2
4,096	0.0792	0.0359	2.2

Note what the coordinate model does at the bottom of the table: it **plateaus** at about 0.079 and stops improving. More data does not close the gap, because capacity is being spent representing a fact the other model was handed for nothing.

This is Week 4′s data-efficiency result with a sharper ending. There, more data eventually let the unstructured model catch up. Here it does not, within this budget, because the structure is not a shortcut to something learnable but a genuine reduction of the problem.

MEASURED IN THE LAB

Orientations never seen in training. Train on a narrow band of orientations, test on all of them:

Model	Orientations seen	All orientations	Degradation
coordinate MLP	0.0634	0.3860	**6.1 times worse**
invariant model	0.0475	0.0476	**1.0**

The coordinate model’s error on unseen orientations (0.386) is **worse than simply predicting the average energy**, which would score about 0.330. It has not partially generalised; it has produced confident nonsense.

This is Week 4, Chapter 9′s "positions never seen in training" experiment with rotation substituted for translation, and the same lesson: a symmetry you built in is free everywhere, and a symmetry you learned from data holds only where you had data.

⌕ DEEPER DETAIL: A design trap worth knowing about, from building this experiment

The first version of this experiment measured nothing, and the bug was invisible.

The obvious way to generate "a dataset that only saw some orientations" is to draw the coordinates from an isotropic Gaussian and then rotate by a restricted angle. That does not work: **an isotropic Gaussian is already rotationally symmetric**, so restricting the rotation changes nothing at all. The distribution of arrangements was identical in both conditions, and the experiment dutifully reported a degradation factor of about 1.0 for both models.

The fix was to build arrangements with a definite orientation, three particles at fixed angular positions with random radii, and then apply the global rotation.

The general lesson is worth carrying: **if you want to test whether a model has learned an invariance, your data has to be capable of violating it.** An experiment that cannot fail is not measuring anything, and it will not look broken.

▸ OPEN THE SYMMETRY EXPLORER

Open `interactive/symmetry-explorer.html`, tab 1, and press **Spin**.

Watch the two readouts. The coordinate model's prediction wanders; the invariant model's does not move at all. Neither model is trained: the difference is entirely in **what they read**, and the right-hand panel shows exactly that, with the six coordinates changing while the three distances stay identical to four decimal places.

Then drag the **translation** slider and watch the same thing happen for a different symmetry.

QUIZ WATCH · WEEK 6

Very likely: explain why a symmetry is a stronger prior than a statistical regularity. State the difference between invariance and equivariance, with an example of each. Reproduce and explain the 8.88e-16 result. Explain why the coordinate model plateaus.

15 Equivariance in Practice, and When It Is Wrong

COVERS How real equivariant models are built, and the counterexample that keeps this honest
TIME About 14 minutes
IN BRIEF Sorting distances is a toy. Real systems use architectures whose layers are equivariant by construction. And a false symmetry is a wall, not a handicap.

15.1 How this is done properly

The lab's approach, computing invariant features by hand, works for three particles and does not scale. For a molecule with fifty atoms, you cannot enumerate features and you need the **intermediate representations** to be equivariant, not just the final answer.

Two families dominate:

Invariant message passing. A graph neural network where each atom exchanges messages with its neighbours, and the messages depend only on invariant quantities such as distances and angles. Since every ingredient is invariant, so is every layer. SchNet and DimeNet are the standard examples in chemistry.

Equivariant networks. The internal features are vectors and tensors that **rotate with the input** rather than staying fixed. This keeps directional information, which invariant features throw away, and it matters when you want to predict a force rather than an energy. Forces are vectors: rotate the molecule and the forces rotate too, so the correct property is equivariance, not invariance. E(3)-equivariant networks and their relatives are the machinery, and they are central to modern molecular modelling.

KEY IDEA

Invariance and equivariance are not competitors; they answer different questions.

Predicting a **scalar** like energy: you want **invariance**. Rotating the input must not change the answer.

Predicting a **vector** like force or velocity: you want **equivariance**. Rotating the input must rotate the answer by the same amount.

Getting this backwards produces a model that is confidently unable to represent its own target, which is Week 4, Chapter 9′s flatten-head failure in a new costume.

15.2 The counterexample

Everything above says symmetry is worth having. The honest question is what happens when it does not hold.

MEASURED IN THE LAB

From `experiment_wrong_symmetry()`. Add a gravity term to the energy, so that height now matters and the energy genuinely **does** change when the arrangement is rotated.

Model	Error	Why
coordinate MLP	**0.0934**	can represent the height term
invariant model	0.2355	**structurally cannot, at any dataset size**

The invariant model is now two and a half times worse and no amount of data will help. It discarded the heights when it computed distances, and that information is not recoverable downstream.

COMMON MISTAKE

"Building in a symmetry is a safe default, since worst case it just does not help."

It is not safe, and this is the most important sentence in the chapter. A constraint is a **promise about the world**. If the promise is true you get an exact guarantee. If it is false you get a model that cannot represent the truth, no matter how much data or capacity you give it.

This is exactly parallel to Week 4′s "pre-centred digits" result, where a convolution's assumption bought nothing because the problem had already been solved in preprocessing. The difference is that here the failure is worse: not "buys nothing", but "makes the correct answer unreachable".

✓ IN PRACTICE: Checking a symmetry before you impose it

Three steps, all cheap, all worth doing.

1. **State it in plain language.** "Rotating this input should not change the output." If you cannot state it in one sentence, you do not know what you are about to impose.
2. **Test it on the data you have.** Take pairs of examples related by the transformation and check that their labels agree. If your data does not contain such pairs, apply the transformation to an input and ask a domain expert whether the label should change.
3. **Ask what is being thrown away.** An invariant feature map discards something. Write down what. In the gravity example the answer is "absolute height", and once written down it is obvious that the assumption is wrong.

▸ OPEN THE SYMMETRY EXPLORER

Tab 3, "What symmetry costs and buys". Drag the **gravity** slider up from zero and read the panel on the right. At zero the symmetry holds and imposing it is free. Above zero the panel states plainly that an invariant model cannot represent the target at any dataset size.

The left panel is worth a moment too: it counts how much of the input space the symmetry removes. For n particles in a plane, three of the $2n$ raw numbers describe only where the arrangement sits and which way it faces, and carry no information about the arrangement itself.

QUIZ WATCH · WEEK 6

Likely: distinguish when you want invariance from when you want equivariance, with an example of each. Explain the gravity counterexample and why more data cannot fix it. Describe how a real equivariant architecture differs from computing invariant features by hand.

16 Neural ODEs: Learning the Rule, Not the Answer

COVERS Representing the derivative instead of the solution, and what that buys
TIME About 18 minutes
IN BRIEF A PINN learns one trajectory. A neural ODE learns the vector field that generates every trajectory.

16.1 The inversion

A PINN represents the solution directly: the network **is** $u(t)$, and the loss forces it to satisfy the equation. That works beautifully and has one structural limitation. The network has learned **one trajectory**. Change the initial condition and you must train again from scratch.

A neural ODE inverts the arrangement. The network does not represent the solution; it represents the **derivative**:

$$\frac{du}{dt} = f_\theta(u)$$

To get a trajectory, hand f_θ to a numerical integrator and roll forward from wherever you like.

KEY IDEA

Learning a trajectory gives you an answer. Learning a vector field gives you a model, and a model can be asked questions you did not train it on.

This distinction recurs throughout the week and is worth holding onto. It is the same move as Chapter 18′s operators, one level up.

IN PLAIN ENGLISH

The difference between memorising a route and learning to read a map.

Memorise the route from your house to the station and you can make that journey. Ask for a different starting point and you have nothing.

Learn to read the map and you can start anywhere, because you have the **rule** rather than one instance of following it.

A neural ODE learns the map. The arrows in the lab's vector-field figure are that map: each one says "if you are here, this is where you go next", and every possible trajectory is generated by following them.

16.2 How it is trained

Roll the integrator forward inside the computation graph, compare the whole rolled-out trajectory against the data, and differentiate **through the integrator**. The lab uses Runge-Kutta 4 with every step in the graph, which is the straightforward approach and keeps the code readable.

⌕ DEEPER DETAIL: The adjoint method, and why the original paper needed it

Differentiating through the solver means storing every intermediate step for the backward pass. With forty steps that is cheap. With ten thousand it is not, and this was the obstacle the 2018 neural ODE paper had to clear.

Their solution is elegant: instead of storing the forward trajectory, solve a **second** ODE backwards in time to recover the gradients. Memory becomes constant in the number of steps rather than linear.

The trade is honest and worth knowing: the adjoint method uses less memory and can be less numerically accurate, because reconstructing the forward trajectory backwards accumulates its own error. Modern implementations offer both, and for problems the size of this lab's, storing everything is simpler and better.

16.3 What it buys

MEASURED IN THE LAB

Rolling out past the training window. Train on 20 steps, then keep integrating to 60.

Model	20 steps (trained)	60 steps	Degradation
direct map $(t, u_0) \to u$	0.0278	0.4676	**16.8 times worse**
neural ODE	0.0207	0.0351	**1.7**
structured ODE	0.0128	0.0210	**1.6**

The direct map is a fair baseline: it takes the initial state as an input, so it **can** represent the whole family of trajectories. Its limitation is subtler. It has learned a function on the box of values it was trained on, and outside that box it has no opinion worth having. A vector field is defined everywhere and can be integrated for as long as you ask.

MEASURED IN THE LAB

Initial conditions outside the training box:

Model	Starts seen	Starts outside the box	Times worse than best
direct map	0.0246	0.2310	2.2
neural ODE	0.0132	0.1768	1.7
structured ODE	**0.0077**	**0.1057**	**1.0**

Report this one carefully. **All three degrade outside the box**, and the honest conclusion is that structure buys a smaller error rather than immunity. Rolling a learned field forward from a state it never visited means evaluating it where it was never fitted, and no amount of structure removes that.

QUIZ WATCH · WEEK 6

Very likely: explain the difference between learning a trajectory and learning a vector field, and what each permits. Explain why the direct map degrades 16.8 times when rolled out and the neural ODE 1.7 times. State what the adjoint method solves and what it costs.

17 Structured Dynamics: Learning Only the Unknown Half

COVERS	Building the known part of the physics into the vector field itself
TIME	About 12 minutes
IN BRIEF	If half the equation is a definition, do not make a network discover it.

The lab's third model is the most instructive, and its idea is embarrassingly simple.

For a mechanical system with position x and velocity v, the vector field has two components:

$$\frac{dx}{dt} = v, \qquad \frac{dv}{dt} = \text{something}$$

The first line is not physics. It is what **velocity means**. Handing it to a network to discover from data is spending capacity and examples on a definition.

So impose it, and learn only the second:

```
def field(self, u):
    out = self.net(u)              # the acceleration, learned
    if not self.structured:
        return out
    v = slice_axis(u, 1, 1, 2)     # velocity, read straight from the state
    return concat([v, out], axis=1)
```

⚖️ THE LAW: THE KINEMATIC RELATION

The plain sentence: the rate of change of position is velocity.

Why it is exactly true: it is the definition of velocity, not an empirical claim. Nothing can violate it.

How it is built in: the first half of the vector field is copied from the state rather than predicted.

What it buys: the network learns a one-dimensional function instead of a two-dimensional one, with fewer parameters. In the lab it is the most accurate model in every experiment.

MEASURED IN THE LAB

Across the two neural ODE experiments, the structured model wins consistently:

Model	Params	Rolled out 3x	New initial conditions
direct map	1,882	0.4676	0.2310
neural ODE	1,842	0.0351	0.1768
structured ODE	**1,801**	**0.0210**	**0.1057**

Fewest parameters and best on every measure. That is the week's thesis in one row: the structured model is not more powerful, it is more **informed**, and being told half the answer beats having to discover it.

WHY THIS MATTERS

This generalises far past mechanics, and it is the most portable idea in the chapter.

In almost every scientific modelling problem, part of the system is understood and part is not. Reaction kinetics may be known while the diffusion coefficient is not. Structural dynamics may be known while

the damping mechanism is not. The pattern is always available: **write down the part you know as fixed structure, and let the network learn only the residual.**

This has a name in the literature, **universal differential equations** or hybrid modelling, and it is arguably more useful in practice than a pure PINN, because it matches how scientific knowledge actually arrives: partial, not total.

› OPEN THE PINN SANDBOX

The sandbox trains a PINN rather than a neural ODE, but the third tab makes the relevant point. Set the assumed damping away from its true value and watch what a **wrong** piece of built-in structure does. In a structured neural ODE the built-in half is a definition and cannot be wrong; in a PINN the built-in equation can be, and Chapter 23 is about exactly that.

QUIZ WATCH · WEEK 6

Likely: explain what a structured neural ODE imposes and why that part cannot be wrong. Explain why the structured model has the fewest parameters and the best accuracy. Give an example from another domain of splitting known from unknown dynamics.

18 Neural Operators: Solving a Whole Family at Once

COVERS	Learning the map from a problem to its answer, rather than one answer
TIME	About 18 minutes
IN BRIEF	A PINN is trained for one problem. Change a coefficient and you start again. An operator learns the map, so a new problem becomes a forward pass.

18.1 The limitation being fixed

A PINN solves one problem: one equation, one set of coefficients, one initial condition. Change the damping from 0.2 to 0.25 and the trained network is worthless. On this laptop that costs another twenty seconds; on a realistic three-dimensional PDE it costs hours.

That is often unacceptable, because a great many real tasks involve solving the **same** equation thousands of times with different coefficients:

- **design optimisation**: try ten thousand wing shapes;
- **uncertainty quantification**: propagate a distribution over an unknown coefficient;
- **control**: re-solve at every timestep as conditions change;
- **inverse problems**: search over coefficients to match a measurement.

KEY IDEA

A neural operator learns a map from **a function or a parameter** to **a function**:

$$G : a \longrightarrow u$$

Train it once on many solved instances. Afterwards, a new instance is a forward pass, which is microseconds.

This is the same inversion as Chapter 16, one level higher. A neural ODE learns the rule that generates trajectories instead of one trajectory. An operator learns the rule that generates **solutions** instead of one solution.

18.2 DeepONet, which is two networks and a dot product

The architecture is far simpler than the name:

$$G(a)(t) = \sum_{i=1}^{p} \underbrace{\text{branch}_i(a)}_{\text{coefficients}} \times \underbrace{\text{trunk}_i(t)}_{\text{basis functions}} + b$$

- The **trunk** reads the query location and outputs p numbers: a set of **basis functions** of t.
- The **branch** reads the problem parameter and outputs p numbers: **how much of each basis function this particular problem needs**.

IN PLAIN ENGLISH

This is a learned Fourier series.

A Fourier series writes any function as a weighted sum of sines and cosines. The sines and cosines are fixed in advance and you compute the weights.

DeepONet does the same thing with one change: **the basis is learned** from the family of problems you actually care about, rather than handed to you. So instead of needing a hundred sine terms to represent your solutions, you may need twenty learned ones, because they were shaped to fit exactly this family.

MEASURED IN THE LAB

From `experiment_operator()`. Train one operator on 40 solved instances of the damped oscillator with different damping coefficients. Then ask it for 20 damping values it has never seen.

Approach	Cost per new problem	Error on unseen parameters	One-off cost
neural operator	**0.25 ms**	0.0379	4 s
a fresh PINN each time	21 s	**0.0006**	none

The operator answers a new problem about 83,000 times faster and is roughly 60 times less accurate.

Both halves of that sentence matter and neither should be dropped. For one problem, train a PINN. For a design loop over ten thousand parameter values, 21 seconds each is 58 hours and 0.25 ms each is 2.5 seconds, and the accuracy is often good enough to guide a search that is refined properly at the end.

EVERYDAY ANALOGY

The difference between a solicitor and a book of standard contracts.

Ask a solicitor to draft one agreement and you get something exactly right for your situation, at a cost, after a delay.

A book of templates was expensive to write once and gives you an answer immediately, and it is not quite right for any particular case.

Which you want depends entirely on whether you need one agreement or ten thousand. Neither is the better tool in general, and a great deal of confused writing about neural operators comes from forgetting to say which situation is being discussed.

QUIZ WATCH · WEEK 6

Very likely: explain what a neural operator maps between and why that differs from a PINN. Describe DeepONet's branch and trunk in one sentence each. State the measured trade between speed and accuracy and say when each side wins.

19 The Operator Landscape

COVERS	FNO, DeepONet, and what "resolution independence" actually means
TIME	About 14 minutes
IN BRIEF	Two architectures, one goal. The Fourier Neural Operator takes a different route and dominates on regular grids.

19.1 The Fourier Neural Operator

The FNO reaches the same goal by a different road, and the idea is worth understanding because it is unusually elegant.

A convolution in space is a **multiplication** in frequency space. That is the convolution theorem, and it means a global convolution, which would be expensive to compute directly, becomes cheap:

1. Fourier transform the input, with an FFT.
2. Multiply by learned weights, one per frequency.
3. Transform back.

KEY IDEA

The FNO's operation is a **global** convolution with a learned kernel, computed cheaply in frequency space. Every point influences every other point in a single layer, which is what an operator needs, since changing an initial condition anywhere changes the solution everywhere.

Compare Week 4′s attention, which also gives every position access to every other, at quadratic cost. The FNO gets global mixing at $O(n \log n)$ instead, by exploiting the structure of a regular grid. It is a good example of an assumption buying efficiency: attention assumes nothing about the arrangement of its inputs and pays for it; the FNO assumes a grid and is rewarded.

Resolution independence is the FNO's headline property and it is often stated loosely. The learned weights live in frequency space, and the low frequencies of a function do not depend on how finely you sampled it. So an FNO trained on a 64 by 64 grid can be evaluated on a 256 by 256 grid, because the parameters describe the **function**, not the pixels. A CNN trained at one resolution has no such property, as Week 4, Chapter 9 discussed for global pooling.

The honest qualification: it is resolution independent for the frequencies it was trained on. Fine structure that was invisible at the training resolution will not appear by magic at a higher one.

19.2 Choosing between them

	DeepONet	FNO
Input	any parameter or function, sampled anywhere	a function on a regular grid
Query points	arbitrary, one at a time	the whole grid at once
Geometry	handles irregular domains naturally	prefers regular grids
Efficiency	good	excellent on grids, via the FFT

Typical use	scattered sensors, irregular geometry	fluid dynamics, weather, images
Implementable in numpy in forty lines	**yes, and the lab does**	not comfortably

Weather, which is where this became serious

The most consequential application of operator learning so far is weather forecasting, and it is worth knowing about because it changed how the field is viewed.

Numerical weather prediction has been a triumph of classical computational physics for fifty years: solve the governing equations on a very large grid, on a very large computer, for several hours.

From about 2022, operator-style models (FourCastNet, GraphCast, and others) began producing forecasts of comparable skill in **seconds** on a single machine, having been trained on decades of reanalysis data. Some are now run operationally alongside the physics-based models.

Two lessons, and the second is easy to miss. First, the speed-up is not incremental: it is four or five orders of magnitude, which changes what is possible, since you can now run a thousand-member ensemble to quantify uncertainty. Second, these models are **trained on the output of physics-based simulations**, so they have not replaced the physics; they have compressed it. The classical solvers are still needed to produce the training data, and that is the honest shape of the achievement.

› OPEN THE PINN SANDBOX

Not directly, but the comparison in Chapter 18′s table is worth reproducing mentally while using the sandbox. Each time you press **Train**, you are paying the per-problem cost that an operator amortises. Change the damping slider and press Train again: that second training run is the cost an operator exists to avoid.

QUIZ WATCH · WEEK 6

Likely: explain how the FNO uses the Fourier transform and why that gives global mixing cheaply. State what resolution independence means and its honest limitation. Give one situation favouring DeepONet and one favouring FNO.

20 Diffusion Models as Non-Equilibrium Thermodynamics

COVERS	Where image generators came from, and why the physics framing makes them obvious
TIME	About 20 minutes
IN BRIEF	The forward process is given by physics and learned by nobody. That is what turns an intractable generative problem into a sequence of ordinary regressions.

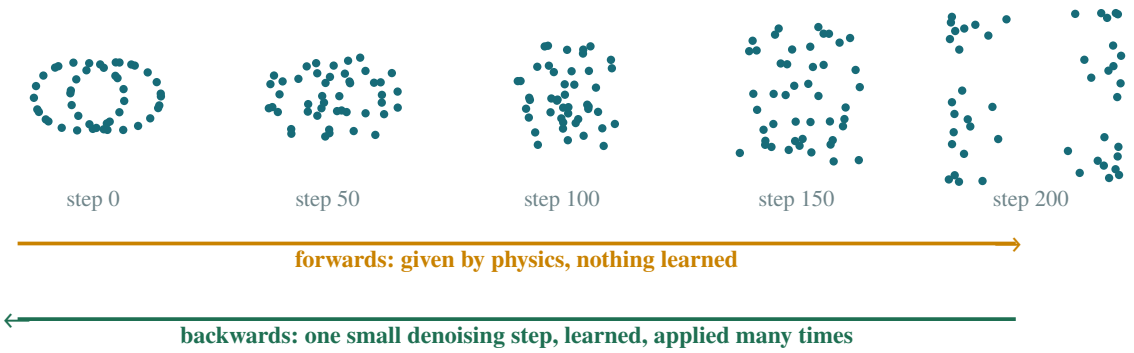
Every image generator you have used descends from a 2015 paper titled **Deep Unsupervised Learning using Nonequilibrium Thermodynamics**. The title is not a metaphor. The method is taken from statistical physics, and understanding it that way makes the algorithm obvious rather than mysterious.

20.1 The physical picture

Put a drop of ink in a glass of water. Molecular collisions push the ink about at random, the sharp structure dissolves, and after long enough the glass holds a featureless uniform solution. That is diffusion driving a system to **equilibrium**, and it destroys information: from the final glass you cannot tell where the drop was.

The forward process here is exactly that, applied to a dataset. Add a little Gaussian noise, repeatedly. After enough steps the data is indistinguishable from an isotropic Gaussian.

Now the trick. Each individual step added only a **little** noise, so each individual step is nearly reversible. Learn that one small reversal, apply it many times, and you have run the diffusion backwards: start from pure noise, denoise repeatedly, and arrive at something that looks like the data.



⚖ THE LAW: DIFFUSION TO EQUILIBRIUM

The plain sentence: random jostling destroys structure at a known, chosen rate, and ends in a state that has forgotten everything.

How it is built in: the forward process is **specified**, not learned. The noise schedule is written down in advance and never trained.

What it buys: the hard problem, “map noise to data in one leap”, is replaced by hundreds of easy problems, “undo a little noise”, each of which is an ordinary squared-error regression.

What it costs: generation needs many sequential network calls, which is why diffusion models are slow to sample and why fast samplers are their own research area.

MEASURED IN THE LAB

The forward process reaching equilibrium, from `experiment_equilibrium()`. Nothing here is learned:

Forward step	Fraction of signal left	Distance to a standard Gaussian
0	0.9999	0.0495
10	0.9912	0.0443
40	0.8816	0.0198
100	0.4617	0.0034
199	0.0466	0.0030

By step 199 the data is statistically indistinguishable from the equilibrium distribution. That is the state the reverse process starts from, and it is why you can begin generation from pure noise.

20.2 Why the structure makes the learning easy

Compare with the alternative. A generative adversarial network learns the whole map from noise to data in one leap, with nothing given. It works, and it is notoriously unstable to train, because two networks are competing and the equilibrium between them is delicate.

A diffusion model breaks the same journey into hundreds of tiny steps **whose forward direction is known exactly**, and each step is a plain regression with a plain squared-error loss. There is no adversary, no discriminator, and no instability.

KEY IDEA

Stability came from structure, not from a better optimiser.

This is the week's thesis appearing somewhere you would not expect it. The physical prior here is not a differential equation constraining the answer; it is a known **process** used as scaffolding, and it converts a hard problem into an easy one.

MEASURED IN THE LAB

Sample quality, from `experiment_diffusion()`, on a two-crescent distribution. The **floor** is the energy distance between two independent samples of the real data, so nothing can meaningfully score below it.

Model	Energy distance	Times the floor
pure noise (the equilibrium)	0.0468	11.18
a fitted Gaussian	0.0297	7.09
diffusion, 10 reverse steps	0.0120	2.87
diffusion, 50 reverse steps	**0.0038**	**0.91**
diffusion, 200 reverse steps	0.0063	1.50
two real samples (the floor)	0.0042	1.00

At 50 reverse steps the generated samples are **statistically indistinguishable from real data**, while a fitted Gaussian is seven times worse. The Gaussian had the right mean and the right covariance and completely the wrong shape.

Note that the diffusion model had no more information than the Gaussian did. It simply had the forward process handed to it.

◗ OPEN THE DIFFUSION AND THERMODYNAMICS EXPLORER

Open `interactive/diffusion-thermodynamics-explorer.html` and press **Run forwards** on tab 1. Watch the two crescents dissolve into a featureless cloud, with the signal-fraction curve tracking how much structure is left.

Then go to tab 2 and press **Generate**. The same process, run backwards, starting from noise.

One honest note the page makes itself: because the data there is a mixture of Gaussians, the score can be written down **exactly**, so that page uses the exact score rather than a trained one. That separates two questions which are usually tangled: what the method does, and how well a network approximates it. The Python lab does the other half, training a real denoiser on data with no closed form.

QUIZ WATCH · WEEK 6

Very likely: describe the forward and reverse processes and say which is learned. Explain why breaking the problem into many small steps makes it stable, comparing with a GAN. Explain the physical analogy and what "equilibrium" means here.

21 The Score, and What Was Borrowed

COVERS	The mathematics underneath diffusion, and its considerable prior history
TIME	About 14 minutes
IN BRIEF	Reversing a diffusion requires the gradient of the log density. Physicists have been using that to sample for a century.

21.1 The score

To run a diffusion backwards you need to know, at every point, which direction leads towards higher probability. That is the **score**:

$$s(x) = \nabla \log p(x)$$

the gradient of the log of the density. It points towards where the data is denser.

IN PLAIN ENGLISH

Imagine a landscape where valleys are where the data lives. The score is the direction of steepest descent into the nearest valley, evaluated wherever you are standing.

To generate a sample: start anywhere, step downhill a little, add a small random jolt, repeat. You end up in a valley, and the jolts mean you end up in **a** valley in proportion to how big it is, rather than always the nearest one.

That procedure is called **Langevin dynamics**, it is how physicists have sampled from Boltzmann distributions since the early twentieth century, and it is precisely what the reverse process of a diffusion model does.

21.2 The connection to the training objective

The model is not trained to output the score. It is trained to predict **the noise that was added**, with a squared-error loss. Those turn out to be the same thing up to a scaling, which is the result that ties the practical algorithm to the theory:

$$s(x_t) \approx -\varepsilon_\theta \frac{x_t, t}{\sqrt{1-\bar{\alpha}_t}}$$

So a network trained on the most ordinary objective imaginable, “guess the noise”, is learning a physical quantity with a century of theory behind it.

⌕ DEEPER DETAIL: Why predicting the noise rather than the clean sample

They are equivalent by algebra: knowing x_t and either one gives you the other. But the choice matters in practice, for a reason worth recognising.

The **noise** target has unit variance at every step, by construction. So one network with one learning rate handles every noise level equally well.

The **clean sample** target has a scale that changes by orders of magnitude across the schedule: early on it is nearly the data, late on it must be reconstructed from almost nothing.

This is a reparameterisation that makes the optimisation behave, and it is in the same family as Week 3′s feature scaling and Chapter 9′s input scaling. Nothing about the model changed; the target was rewritten so that a single learning rate suits it.

21.3 The vocabulary that came from physics

Term	What it means here	Where it came from
diffusion	the forward noising process	Brownian motion, Einstein 1905
equilibrium	the distribution the forward process ends at	statistical mechanics
Langevin dynamics	step along the score, add noise, repeat	Langevin 1908, for particles in a fluid
Fokker-Planck	the PDE the noising process obeys	statistical physics, 1910s
score	$\nabla \log p$	statistics (Fisher), reused
annealing	going from high to low noise	metallurgy, via simulated annealing
temperature	how much noise is injected during sampling	thermodynamics

WHY THIS MATTERS

This table is the argument of the week in one place, and it is worth dwelling on.

Nobody invented this vocabulary for machine learning. The most commercially significant generative modelling technique of the last decade is an application of century-old statistical physics, found by people who knew the physics and noticed it applied.

The transferable lesson is not about diffusion. It is that **knowing the science adjacent to your problem is a competitive advantage in machine learning**, not a distraction from it. Weeks 3 and 4 were about the machinery of learning. This week is a sustained argument that the machinery is not the whole job, and this chapter is the sharpest example of it.

› OPEN THE DIFFUSION AND THERMODYNAMICS EXPLORER

Tab 3, “The score, and why it is physics”. The arrows are the score field, computed exactly. Drag the step slider from 199 down towards 0 and watch the field sharpen.

Early in the reverse process the density is nearly Gaussian and the arrows all point roughly inward, so coarse structure is recovered first. Late in the process they resolve fine detail. A diffusion model builds an image the way a photograph develops: the big shapes arrive before the edges, and this tab shows why that is forced by the mathematics rather than chosen.

QUIZ WATCH · WEEK 6

Likely: define the score in words and say what it is used for. Explain the connection between predicting noise and estimating the score. Name two pieces of vocabulary diffusion models borrowed from physics and say what each meant originally.

22 Thermodynamics-Informed Models

COVERS	Constraints that are inequalities rather than equations
TIME	About 12 minutes
IN BRIEF	Some physics says "this can only increase". That is imposable too, and often more cheaply than an equation.

Not all known physics is an equation. The second law of thermodynamics is an **inequality**: entropy never decreases in an isolated system. Concentrations are never negative. Masses are never negative. Probabilities lie in $[0, 1]$.

These are exploitable, and often more easily than a governing equation, because they can be imposed architecturally.

⚖ THE LAW: THE SECOND LAW

The plain sentence: in an isolated system, entropy never goes down.

Why it is exactly true: it is one of the most thoroughly tested statements in science, and unlike a specific governing equation it does not depend on knowing any coefficients.

How it is built in: make the model's entropy production the **square** or the **softplus** of a network output, so it cannot be negative whatever the weights do.

What it buys: a simulation that cannot run backwards in time or spontaneously unmix, which is exactly the kind of nonsense an unconstrained learned dynamics model produces on long rollouts.

22.1 The GENERIC structure, and thermodynamics-informed GNNs

There is a formalism from non-equilibrium thermodynamics, usually called **GENERIC**, which says that the evolution of a physical system splits into two parts: a reversible part that conserves energy, and an irreversible part that produces entropy.

A thermodynamics-informed graph neural network builds that split into the architecture. Particles or mesh nodes become graph vertices; the learned dynamics is decomposed into the two parts; and the structure guarantees energy conservation and non-negative entropy production **by construction**, exactly as Chapter 10′s hard constraint guaranteed an initial condition.

The payoff is the same as everywhere else in this week: the model cannot produce a physically impossible trajectory, so long rollouts stay stable where an unconstrained model drifts into nonsense.

✓ IN PRACTICE: Cheap architectural guarantees worth knowing

Most of these cost one line and are worth reaching for before anything more elaborate.

You know	Do this	Result
output is non-negative	final layer softplus or square	cannot be negative, ever
output is in $[0, 1]$	final layer sigmoid	cannot leave the range
outputs sum to a fixed total	softmax times the total	mass conserved exactly
output is symmetric in two inputs	average the two orderings	symmetry exact
a rate can only increase	integrate a non-negative output	monotone by construction

output must be zero at a boundary	multiply by a function vanishing there	Chapter 10′s trick

Every one of these is a **hard** constraint. None can be violated by an unlucky optimiser, a bad loss weight or an outlier in the data, and none requires a penalty term to be balanced.

WHY THIS MATTERS

This is the most immediately usable chapter in the week for a working engineer, and the least glamorous.

You will often not have a governing equation. You will almost always have some inequality you are certain of: a concentration that cannot be negative, a probability that must sum to one, a temperature that cannot go below absolute zero, a quantity that can only accumulate. Imposing those is cheap, safe, and removes whole classes of embarrassing failure.

It is also the easiest thing in this week to explain to a non-specialist stakeholder, which matters more than it should.

QUIZ WATCH · WEEK 6

Likely: give an example of physical knowledge that is an inequality rather than an equation, and say how to impose it architecturally. Name three cheap architectural guarantees and what each enforces. Explain what a thermodynamics-informed model guarantees on long rollouts.

23 When Physics-Informed Fails

COVERS	The honest chapter. A wrong law is worse than no law, and it does not look like a failure
TIME	About 20 minutes; the most important chapter for anybody who will use this
IN BRIEF	Every other chapter says physical priors are powerful. This one says what happens when the physics is wrong, and the answer is worse than losing the benefit.

Week 4 had a result that mattered more than its winning numbers: on pre-centred digits, the convolutional network **lost**, because the assumption had already been satisfied in preprocessing. Week 5′s equivalent is sharper and more consequential.

23.1 A wrong equation

MEASURED IN THE LAB

From `experiment_wrong_physics()`. The true system has damping $c = 0.2$. All models get real, noisy measurements from the true system. The physics-informed models are additionally told a value of c, correctly or otherwise.

Model	c assumed	True error	Physics loss it reports
data-only (no physics)	n/a	0.0215	n/a
physics + data, correct	0.2	**0.0142**	5.79e-04
physics + data, 2x too high	0.4	0.0499	4.67e-03
physics + data, 4x too high	0.8	**0.1455**	2.28e-02

With the correct physics, the model beats the data-only baseline. With the damping wrong by a factor of four, it is **seven times worse than assuming nothing at all**.

Now read the last column, which is the part that should worry you. The physics loss stays small throughout: 2.28e-02 is a perfectly respectable-looking number. Nothing on a training dashboard goes red. The model is confidently, quietly wrong, and it is wrong because it was told something false with great authority.

KEY IDEA

A wrong physical prior is worse than no physical prior, and it is more dangerous than a wrong architectural prior.

Week 4′s convolution, misapplied, gave a model that underperformed. This gives a model that underperforms **and looks healthy**, because the quantity you would naturally monitor, the residual of the equation you supplied, is measuring agreement with your mistake.

You cannot detect this by watching the loss. You have to check against something outside the model.

IN PLAIN ENGLISH

A confident expert with an out-of-date fact is more dangerous than an honest novice.

The novice says “I am not sure” and you go and check. The expert gives you a precise, well-reasoned answer built on the wrong premise, and every downstream calculation inherits the error without anybody noticing, because everything is internally consistent.

> A physics-informed model with the wrong equation is the confident expert. Its internal consistency is exactly what makes the error invisible.

23.2 The four ways this goes wrong

One: the equation is wrong. A missing term, an unmodelled mechanism, a coefficient measured badly. Above.

Two: the symmetry does not hold. Chapter 15′s gravity term. The model is structurally unable to represent the truth, and no dataset can rescue it.

Three: the loss terms are misweighted. Chapter 10′s 400-fold spread. Every version satisfied the conditions and only some solved the equation.

Four: the collocation points are unrepresentative. Chapter 11. A small residual at ten points says nothing about the eleventh.

> **MEASURED IN THE LAB**
>
> And a fifth, from Chapter 12, which is less obvious. When the physics is **exactly right**, adding noisy data actively hurts:
>
> | Model | Error |
> |---|---|
> | physics only, no data | **0.00135** |
> | physics + 8 noisy measurements | 0.02665 |
>
> Twenty times worse. The equation plus the initial conditions already determine the answer completely, so the noisy data has nothing to add and can only pull the solution away.
>
> This says something general about hybrid models: **the weight on each term should reflect your relative confidence in the model and the measurements.** Defaults of 1 and 1 encode the belief that they are equally trustworthy, and that is a claim, not a neutral choice.

23.3 How to check

> **✓ IN PRACTICE: Five checks, in order of cost**
>
> 1. **Refine the collocation set.** Double the points and retrain. If the answer moves, it had not converged.
> 2. **Evaluate the residual at fresh points** that were never trained on. A large residual there means you fitted the collocation set rather than the equation.
> 3. **Check a conserved quantity.** Chapter 12. It is a consequence of the equation that you did not train on, so it is genuine out-of-sample evidence and it costs nothing.
> 4. **Vary the assumed coefficients** and see how much the answer moves. If a 10% change in a coefficient you are unsure of moves the answer a lot, your uncertainty about that coefficient is the dominant error and you should be reporting it.
> 5. **Hold out real measurements** from a region and check there. The most expensive and the most conclusive.
>
> A project that does none of these has a physics-informed model whose physics has never been tested, which is a strange position to be in.

> **⌕ DEEPER DETAIL: Inverse problems, or turning the failure into the method**

There is an elegant response to "I do not know the coefficient": **learn it**.

Make c a trainable parameter alongside the network weights, supply real measurements, and minimise the same combined loss. Gradient descent will adjust both the solution and the coefficient until the physics and the data agree.

This is an **inverse problem**, and it is arguably where PINNs are most valuable, because it is where classical solvers are weakest. A finite-element solver needs the coefficients as input; a PINN can treat them as unknowns and infer them from sparse noisy observations, using exactly the same machinery.

The applications are substantial: estimating tissue properties from medical images, subsurface permeability from well data, unknown reaction rates from concentration measurements. In each case the equation's **form** is known and its **parameters** are not, which is by far the most common scientific situation.

Note the connection to Chapter 17′s structured neural ODE. Both split the known from the unknown; the difference is only which part is which.

QUIZ WATCH · WEEK 6

Very certain to appear in some form: explain why a wrong physical prior is worse than none, using the measured numbers. Explain why the physics loss does not reveal the problem. Name three checks that would detect it. Explain what an inverse problem is and why PINNs suit it.

24 Choosing a Science-Grounded Method

COVERS A decision procedure, and how to defend the decision
TIME About 12 minutes
IN BRIEF Match the method to the kind of knowledge you have, not to the method's fame.

If you have	Reach for	Because
a governing equation, one problem instance	a PINN	it needs no data and gives you the solution and its derivatives
a governing equation, thousands of instances	a neural operator	amortise the cost; Chapter 18′s 83,000-fold speed-up
a governing equation and a **standard** domain	**a classical solver**	faster, more accurate, with error bounds; Chapter 13
an equation with unknown coefficients	a PINN as an inverse problem	treat the coefficient as a trainable parameter
partial dynamics, some known	a structured neural ODE	impose the known half, learn the rest; Chapter 17
an exact symmetry	an equivariant or invariant model	exact guarantee, better data efficiency; Chapter 14
an inequality you are sure of	an architectural constraint	one line, cannot be violated; Chapter 22
a known corruption process	a diffusion model	use the known process as scaffolding; Chapter 20
no reliable physics	**ordinary machine learning**	a guessed law is worse than none; Chapter 23

24.1 The three questions

1. **What do I actually know, and how certain am I?** Not "what equation is in the textbook" but "what am I confident holds for this system, with these coefficients, in this regime". Write it as a sentence.
2. **Is it exactly true, approximately true, or a guess?** Exactly true means impose it hard. Approximately true means impose it softly and weight it by your confidence. A guess means do not impose it, and consider learning it instead.
3. **How much data do I have, and how good is it?** Scarce and noisy favours strong priors. Abundant and clean reduces the benefit and increases the risk that a slightly wrong prior does more harm than good.

✓ IN PRACTICE: Always fit the honest baseline

Week 3 asked whether a simpler model would have done. Week 4 repeated it. It applies with more force here, because physics-informed methods are more work and more fragile than what they replace.

Before building a PINN, fit the data-only model. Before either, ask whether a classical solver would do the job in a second. You need to know what the complicated thing bought you, in a number.

This lab supplies the uncomfortable example: on the pre-centred digits of Week 4 the plain network won, and here, when the assumed damping is wrong by a factor of four, the data-only model wins by seven times. Neither would have been discovered by somebody who assumed the sophisticated method was better.

QUIZ WATCH · WEEK 6

Very likely as an applied question: given a described problem, choose a method and justify it by naming the knowledge available and its reliability. Expect at least one case where the right answer is “a classical solver” or “ordinary machine learning”.

25 Responsibility: Models That Cannot Lie About Physics

COVERS	Week 2′s argument, arriving through the equations
TIME	About 12 minutes
IN BRIEF	A hard constraint is a guarantee you can write into a specification. A soft penalty is a hope.

Week 2 argued that a loss function is a values statement. Week 3 added that an unchosen hyperparameter is a decision made by nobody. Week 4 said an architectural assumption is a claim about the world made on your behalf. Week 5 completes the sequence with something more useful than a warning: a way to make guarantees.

25.1 The guarantee you can actually make

Most claims about a machine learning model are statistical: it was 94% accurate on a test set drawn from a distribution you hope resembles deployment. That is a claim about the past, not a promise about the future.

A hard constraint is different in kind.

KEY IDEA

"This model's output satisfies $u(0) = 1$" is not a statistical claim. It is a property of the function, true for every input, at every point in training, for any weights, verifiable by reading the code.

You can put that in a specification. You can put it in a safety case. You cannot do either with "the validation loss was low".

That difference matters wherever a model touches a physical system. A controller that cannot command a negative flow rate is safer than one that was trained not to, because the first cannot fail and the second can fail on an input nobody anticipated.

⚑ CASE FILE: When the reliability is the whole point

Consider a model estimating the remaining safe load on a structure from sensor data, trained on measurements from normal operating conditions.

A data-only model interpolates its training distribution well. Asked about a loading regime outside that distribution, it produces a confident number with no basis, and the failure is silent: nothing in the output distinguishes an interpolation from an extrapolation.

Chapter 8′s measurement is the relevant one. Outside the data region the data-only model's error grew **27.6 times** while the physics-informed model's did not grow at all. The reason is that structural mechanics still applies outside the region where sensors happened to be placed, and one of the two models was told.

The responsible position is not "use a PINN and relax". It is: know which of your model's outputs are backed by a physical constraint and which are backed only by resemblance to training data, and say so plainly to whoever is relying on them.

✓ IN PRACTICE: What to put in a model card

For any science-grounded model, document:

1. **What physics is imposed**, as a plain sentence, and **how**: hard constraint or soft penalty. These carry different strengths of guarantee and should never be reported as if they were the same.
2. **What is assumed but not verified.** Which coefficients were taken from a handbook rather than measured on this system.

3. **Where the constraint stops applying.** Every equation has a regime of validity. A model that silently extrapolates past it is exactly as dangerous as a data-driven one.
4. **What was checked.** The five checks in Chapter 23, and their results.
5. **The sensitivity to assumed coefficients.** If a 10% error in a coefficient moves the answer substantially, that is the dominant uncertainty and it belongs in the report.

WHY THIS MATTERS

There is a temptation to treat "physics-informed" as a badge that ends the conversation about trust. It does not, and Chapter 23 is the reason: the failure mode of a wrong physical prior is a confident wrong answer with a healthy-looking loss.

What the methods in this week genuinely provide is **narrower and more valuable**: a specific, checkable, statable set of properties your model cannot violate. That is a real contribution to a safety argument, and it is worth considerably more than an accuracy figure, provided you are precise about which properties are guaranteed and which are merely hoped for.

QUIZ WATCH · WEEK 6

Likely as a discussion prompt: explain the difference between a statistical claim and a structural guarantee about a model. Explain why "we used a physics-informed model" is not by itself a trust argument. State three things you would document.

26 A Field Guide: Seven Questions

COVERS The week, operationalised
TIME About 10 minutes, then use it
IN BRIEF Week 1 for a headline, Week 2 for anything you ship, Week 3 for anything you train, Week 4 for anything you design. These are for anything that claims to know physics.

1. **What do I know, stated as a sentence a non-specialist would understand?** If you cannot say it plainly, you cannot check it. Chapter 5.
2. **Is it exactly true, approximately true, or a guess?** Exact means impose it hard. Approximate means weight it by confidence. A guess means do not impose it. Chapters 10, 23.
3. **Is it imposed as a hard constraint or a soft penalty?** These carry very different guarantees. Hard means the model cannot violate it; soft means it probably will not, subject to a weight nobody can derive. Chapter 10.
4. **Where is the loss actually evaluated, and what does that leave unchecked?** Collocation points tell you about the points you chose. Chapter 11.
5. **What consequence can I check that I did not train on?** Conservation, a symmetry, a bound. This is your only genuine out-of-sample evidence when you have no held-out data. Chapter 12.
6. **What happens if my physics is wrong by 20%?** Vary it and see. If the answer moves a lot, that uncertainty dominates everything else you are reporting. Chapter 23.
7. **Would a classical solver, or ordinary machine learning, have done this better?** Chapters 13 and 24. Sometimes the answer is yes, and knowing that is the mark of understanding the field rather than its enthusiasm.

TRY IT YOURSELF

Standing exercise: answer all seven for any science-grounded model you build, in under 250 words, before reporting a result. By Week 12 this is a marked section of your critical brief.

27 The Revision Core: The Whole Week on Two Pages

27.1 The one sentence

A physical law is a stronger prior than any statistical regularity, because it is exactly true rather than approximately true; the payoff is largest where data is scarcest, and a wrong law is worse than no law.

27.2 The six methods

Method	The idea	The catch
PINN	put the equation's residual in the loss, evaluated where there is no data	one problem per training run; a small loss proves little; collocation scales badly
Neural operator	learn the map from problem to solution, so a new instance is a forward pass	less accurate on any single instance; needs solved instances to train on
Neural ODE	learn the vector field, integrate it, so you learn the rule not the answer	still degrades outside the region the field was fitted in
Equivariant model	build an exact symmetry into the features or the layers	if the symmetry is false the model cannot represent the truth at any dataset size
Diffusion model	use a known corruption process as scaffolding; learn only to undo one small step	many sequential steps to sample
Thermodynamics-informed	impose inequalities such as non-negative entropy production, architecturally	needs the right decomposition; less general than it sounds

27.3 The numbers to know

Claim	Number
Autodiff engine, worst error against exact calculus to third order	1.22e-16
PINN on the oscillator, observations used	**zero**
PINN error against data-only with 64 measurements	0.00135 against 0.02648, about 20 times better
Extrapolation beyond the data: data-only against physics	27.6 times worse, against 1.0
Heat equation, PINN with no data against data-only with 128	0.00022 against 0.01741, 79 times better
Energy drift: data-only, hybrid, physics-only	16.3%, 5.1%, **0.2%**
Initial condition: soft penalty spread across weights	0.00154 to 0.66444, over 400 times
Hard constraint, measured $u(0)$	1.00000 exactly
Wrong damping by 4x: physics against data-only	0.1455 against 0.0215, **seven times worse**

Rotational invariance error: coordinate against invariant	1.76e-01 against **8.88e-16**
Unseen orientations: coordinate against invariant	6.1 times worse, against 1.0
False symmetry (gravity term)	invariant model 0.2355 against coordinate 0.0934
Neural ODE rolled out 3x: direct map against structured	16.8 times worse, against 1.6
Operator against a fresh PINN per problem	83,000 times faster, 60 times less accurate
Diffusion at 50 reverse steps, against the sampling floor	0.91 times the floor, against 7.09 for a Gaussian
Physics exactly right: adding noisy data	made it **20 times worse**

27.4 PINNs

Loss is the mean squared residual of the equation, evaluated at **collocation points** that carry no data and cost nothing. Requires derivatives of the network with respect to its **input**, hence an autodiff engine whose gradients are themselves differentiable (`create_graph=True`). ReLU is unusable: its second derivative is zero almost everywhere. Use tanh or sine, and scale the inputs. Impose conditions **hard** where you can, by wrapping the network in a form that satisfies them identically; the soft alternative introduces a weight worth a factor of 400. A small loss proves only that the equation holds at the points you picked.

27.5 What you get free

A model satisfying the equation inherits **everything the equation implies**. Energy was in no loss function and was conserved to 0.2% by the physics-only model against 16.3% drift for data-only. This is both the deepest argument for the approach and your best validation tool when you have no held-out data.

27.6 Symmetry

Exactly true, so exactly enforceable. Read invariant features (distances) and rotation cannot change the answer: measured error 8.88e-16, the same number computed twice. **Invariance** for scalars such as energy; **equivariance** for vectors such as force. Buys data efficiency and generalisation to unseen orientations. If the symmetry is false, the model is structurally incapable, not merely handicapped.

27.7 Learning rules rather than answers

A neural ODE learns $du/dt = f_{\theta(u)}$ and integrates it, so it generalises past the training window and to new starting states. A **structured** ODE imposes the half of the field that is a definition ($dx/dt = v$) and learns only the acceleration: fewest parameters and best accuracy in the lab. A neural operator goes one level higher and learns the map from problem to solution.

27.8 Diffusion

The forward noising process is **given by physics and learned by nobody**; only the reversal of one small step is learned. That converts an unstable one-leap generative problem into hundreds of ordinary regressions. The reverse needs the **score**, $\nabla \log p$, which is Langevin dynamics. Predicting the noise rather than the clean sample keeps the target at unit variance across every noise level.

27.9 Ten one-line lessons

1. A law is exactly true; a regularity is usually true. Only one of them gives guarantees.
2. Collocation points are free, which is the great advantage and the characteristic failure.
3. A small physics loss means the equation holds where you checked, and nowhere else.

4. Impose conditions by construction, not by penalty, whenever the domain allows it.
5. You inherit every consequence of the equation, including the ones you forgot to check.
6. A wrong law is worse than no law, and the loss will not tell you.
7. Symmetry is the cleanest prior: exactly true, exactly enforced, exactly zero violation.
8. Learn the rule, not the answer: a vector field beats a trajectory, an operator beats a solution.
9. Known process structure can scaffold a hard problem, which is what diffusion models do.
10. Sometimes the right answer is a classical solver, and saying so is a mark of understanding.

28 Glossary in Plain English

Term	Plain-English meaning	Ch.
Physics-informed neural network	A network whose loss measures how badly it violates a known equation, rather than how far it is from measurements.	8
Residual	What you get when you substitute a candidate answer into the equation. Zero if it is right.	8
Collocation point	A place where the equation is checked. Costs nothing, because no measurement is needed there.	11
Hard constraint	A condition built into the shape of the function, so it holds for any weights.	10
Soft penalty	A condition added to the loss, so it holds only as well as the optimiser and its weight allow.	10
Loss weight (λ)	How much one term counts against another. Not derivable; worth a factor of 400 here.	10
Governing equation	The differential equation the true answer must satisfy.	3, 5
Initial condition	Where the system started. Needed because the equation alone does not pick one answer.	3
Boundary condition	What happens at the edges of the domain.	13
Higher-order autodiff	Differentiation whose gradients are themselves differentiable. What `create_graph=True` gives.	7
Vector-Jacobian product	The only operation reverse-mode differentiation ever performs.	7
Finite difference	Approximating a derivative from nearby values. Accurate enough once, poor twice.	7
Inverse problem	Knowing the equation's form but not its coefficients, and inferring them from data.	23
Conservation law	A quantity the equation implies cannot change. Free validation.	12
Invariance	Transform the input, the output does not change. Right for scalars such as energy.	14, 15
Equivariance	Transform the input, the output transforms the same way. Right for vectors such as force.	15
Symmetry group	The set of transformations that leave the physics unchanged.	14
Neural ODE	A network representing the derivative, integrated to produce trajectories.	16
Vector field	The rule that says, from any state, where the system goes next.	16
Adjoint method	Recovering gradients by solving a second ODE backwards, to save memory.	16
Structured dynamics	Imposing the known half of the equations and learning only the rest.	17

Operator	A map from a function to a function, rather than a number to a number.	18
DeepONet	An operator built as branch times trunk: learned coefficients times learned basis functions.	18
FNO	An operator that mixes globally by multiplying in frequency space.	19
Resolution independence	Trained at one grid spacing, usable at another, because the weights describe the function.	19
Forward process	The known, unlearned corruption that turns data into noise.	20
Reverse process	The learned denoising that turns noise back into data.	20
Noise schedule	How much noise is added at each step. Chosen, not learned.	20
Equilibrium	The distribution the forward process ends at, having forgotten the data.	20
Score	The gradient of the log density. Which way is “towards the data”.	21
Langevin dynamics	Step along the score, add a jolt, repeat. A century-old sampling method.	21
GENERIC	A formalism splitting dynamics into an energy-conserving and an entropy-producing part.	22
Universal differential equation	A model with known terms and learned terms in the same equation.	17

29 Question Bank

Answers are not printed. Every question is answerable from a named chapter.

29.1 Short answer: two to three sentences each

1. State the difference between a statistical regularity and a physical law as priors. (Ch. 1)
2. Explain what "the physics-informed model used zero observations" means and why it is possible. (Ch. 1, 8)
3. State what a physics loss measures and how it differs from a data loss. (Ch. 2)
4. Explain what a collocation point is and why they cost nothing. (Ch. 2, 11)
5. Which parts of the Week 3 training loop change in a PINN, and which do not? (Ch. 2)
6. Read $u' = -ku$ as an English sentence. (Ch. 3)
7. Read the heat equation as an English sentence. (Ch. 3)
8. Why do initial conditions have to be supplied separately from the equation? (Ch. 3)
9. Name three kinds of scientific prior knowledge other than a governing equation. (Ch. 4)
10. Explain what "known process structure" means and give an example. (Ch. 4, 20)
11. Where do physical priors help most, and give two situations where they do not. (Ch. 5)
12. Why does Week 4's "enough data beats structure" argument not apply the same way here? (Ch. 5)
13. Explain why a PINN's loss requires differentiating a derivative. (Ch. 7)
14. What makes an autodiff engine capable of higher-order derivatives? (Ch. 7)
15. Give two reasons finite differences are unsuitable for a PINN's residual. (Ch. 7)
16. Name the PyTorch argument that makes higher-order autodiff work. (Ch. 7)
17. State the PINN recipe in order. (Ch. 8)
18. Explain the extrapolation result: why 1.0 for one model and 27.6 for the other. (Ch. 8)
19. In what sense is a physics loss a regulariser, and how does it differ from weight decay? (Ch. 8)
20. Explain why ReLU cannot be used in a PINN with a second-order equation. (Ch. 9)
21. Name two activation functions suitable for a PINN, and one reason to prefer each. (Ch. 9)
22. Why does input scaling matter more for a PINN than for an ordinary regressor? (Ch. 9)
23. Give two reasons PINN loss surfaces are hard to optimise. (Ch. 9)
24. State the two ways of imposing an initial condition, with one advantage of each. (Ch. 10)
25. The soft-penalty runs all satisfied $u(0)$ to four decimals yet some were useless. Explain. (Ch. 10)
26. Why is a small physics loss not evidence of a correct solution? (Ch. 11)
27. Name two checks that would detect a solution overfitted to its collocation points. (Ch. 11)
28. Explain why a physics-informed model conserves energy without energy in its loss. (Ch. 12)
29. Why did adding noisy data to an exactly-correct physics model make it worse? (Ch. 12)
30. Name the three things that change when moving from an ODE to a PDE. (Ch. 13)
31. Explain the collocation scaling problem, and give one case where it is not fatal. (Ch. 13)
32. Give three situations where a classical solver is the better tool. (Ch. 13)
33. Why is a symmetry a stronger prior than a statistical regularity? (Ch. 14)
34. Distinguish invariance from equivariance, with an example of each. (Ch. 14, 15)
35. Explain why the coordinate model plateaus and never catches the invariant one. (Ch. 14)
36. Explain the gravity counterexample and why more data cannot fix it. (Ch. 15)
37. When do you want an equivariant rather than an invariant model? (Ch. 15)
38. Explain the difference between learning a trajectory and learning a vector field. (Ch. 16)
39. What does the adjoint method solve, and what does it cost? (Ch. 16)
40. What does a structured neural ODE impose, and why can that part not be wrong? (Ch. 17)
41. Explain what a neural operator maps between. (Ch. 18)
42. Describe DeepONet's branch and trunk in one sentence each. (Ch. 18)

43. State the measured trade between an operator and a per-problem PINN. (Ch. 18)
44. How does the FNO get global mixing cheaply? (Ch. 19)
45. What does resolution independence mean, and what is its honest limitation? (Ch. 19)
46. Describe the forward and reverse processes of a diffusion model and say which is learned. (Ch. 20)
47. Why does breaking generation into many small steps make training stable? (Ch. 20)
48. Define the score in words and say what it is used for. (Ch. 21)
49. Why is the network trained to predict the noise rather than the clean sample? (Ch. 21)
50. Name two pieces of vocabulary diffusion borrowed from physics. (Ch. 21)
51. Give an example of physical knowledge that is an inequality, and how to impose it. (Ch. 22)
52. Name three cheap architectural guarantees and what each enforces. (Ch. 22)
53. Explain why a wrong physical prior is worse than no prior. (Ch. 23)
54. Why does the physics loss fail to reveal that the physics is wrong? (Ch. 23)
55. Name three checks that would detect a wrong physical prior. (Ch. 23)
56. Explain what an inverse problem is and why PINNs suit it. (Ch. 23)
57. Explain the difference between a statistical claim and a structural guarantee. (Ch. 25)

29.2 Applied: read the numbers, do the reasoning

1. A PINN reports a physics loss of 3e-05 and a colleague concludes the solution is correct. Give two reasons for caution and two checks you would run. (Ch. 11, 23)
2. Four runs impose the same initial condition with weights 1, 50, 500 and by hard constraint, giving errors 0.0015, 0.322, 0.664 and 0.0014. All four satisfy $u(0)$ to four decimals. Explain. (Ch. 10)
3. A model trained with data on $[0, 4]$ scores 0.018 there and 0.500 on $[4, 10]$; another scores 0.0086 and 0.0087. Identify which is which and explain the mechanism. (Ch. 8)
4. Two models are compared on energy drift: 16.3% and 0.2%. Neither had energy in its loss. Explain the difference. (Ch. 12)
5. A physics-informed model told the wrong damping scores 0.146 while a data-only model scores 0.022, and the physics loss is 2.3e-02. What went wrong, and why was it not caught? (Ch. 23)
6. Rotating an input changes one model's prediction by 1.76e-01 and another's by 8.88e-16. Explain both numbers. (Ch. 14)
7. An invariant model scores 0.236 and a coordinate model 0.093 on a target with a gravity term. Explain, and say what you would change. (Ch. 15)
8. Verify that $u(x,t) = \sin(\pi x)[1 + tN(x,t)]$ satisfies $u(x,0) = \sin(\pi x)$ and $u(0,t) = u(1,t) = 0$ for any N. (Ch. 10, 13)
9. You need solutions of one PDE for 5,000 different coefficient values. A PINN takes 21 s each; an operator takes 4 s to train and 0.25 ms per query but is 60 times less accurate. Recommend an approach and justify it. (Ch. 18)
10. A 3-D time-dependent PDE is to be solved on a regular domain with known coefficients, and a finite-element solver already exists. A colleague proposes a PINN. Respond. (Ch. 13, 24)
11. A colleague proposes making all your model outputs pass through a softplus because "concentrations cannot be negative". Say whether this is a hard or a soft constraint and what it guarantees. (Ch. 22)
12. You have 40 measurements from one turbine and a governing equation whose damping coefficient you are unsure of to within 30%. Describe your approach, including what you would check. (Ch. 23, 24)

29.3 Discussion prompts

1. Week 4 said a wrong architectural prior gives a model that underperforms. Week 5 says a wrong physical prior gives a model that is confidently wrong with a healthy-looking loss. Which is more dangerous in deployment, and what follows for how you would monitor each?

2. Diffusion models are the most commercially significant generative technique of the decade and are an application of century-old statistical physics. What does that suggest about how machine learning specialists should be trained?
3. “Physics-informed neural networks will replace computational fluid dynamics.” Argue both sides using Chapter 13′s scaling table and Chapter 19′s weather example, then commit.
4. A hard constraint gives a guarantee you can put in a safety case; a statistical accuracy figure does not. Is that difference recognised in how machine learning systems are currently regulated, and should it be?
5. Chapter 12 found that adding real measurements to an exactly-correct physics model made it twenty times worse. What does this imply for the common practice of setting all loss weights to 1?

30 Further Reading

Optional. Nothing here is examinable.

Under an hour. Ben Moseley's "So, what is a physics-informed neural network?" is the clearest short introduction and includes runnable code for the damped oscillator this lab uses. Steve Brunton's video series on data-driven science covers neural ODEs and operators with unusually good visual intuition. Lilian Weng's "What are Diffusion Models?" is the standard derivation for Chapters 20 and 21 and is worth reading immediately after them.

An evening. Raissi, Perdikaris and Karniadakis (2019), the paper that named PINNs, is readable and worth reading for its inverse-problem examples as much as its forward ones. Chen and colleagues (2018), "Neural Ordinary Differential Equations", is short and won a best-paper award largely on the adjoint argument in Chapter 16. Sohl-Dickstein and colleagues (2015) is the origin of diffusion models and its thermodynamics framing is much clearer than most later treatments.

A weekend or more. Karniadakis and colleagues (2021), "Physics-informed machine learning" in **Nature Reviews Physics**, is the survey to read if you read only one. Bronstein, Bruna, Cohen and Veličković, **Geometric Deep Learning**, is the systematic treatment of Chapters 14 and 15 and is the most intellectually satisfying item on this list. For the honest counterweight, look for the literature on PINN failure modes, particularly Krishnapriyan and colleagues (2021) on how PINNs fail on stiff and convection-dominated problems: it is the paper Chapter 23 is written in sympathy with.

Appendix: How Week 5 Maps to the Module

Module element	Week 5 connection
LO2 (characterise the training of the main families of model)	Chapters 7 to 22. The training loop is unchanged from Week 3; the loss is what differs, and Chapter 7 supplies the only genuinely new machinery.
Indicative content	Frontier topic extending items 4 and 5: physics-informed networks, neural operators, neural ODEs, equivariant models, diffusion as non-equilibrium thermodynamics, thermodynamics-informed graph networks.
Week 2 (responsible AI)	Chapter 25: a hard constraint is a guarantee you can write into a safety case, which a validation accuracy is not. Chapter 23 is the counterweight.
Week 3 (how machines learn)	The loop is identical. Chapter 7 extends Week 3′s backpropagation to higher order; Chapter 9′s optimisation difficulties are Week 3′s conditioning problems in a harder setting.
Week 4 (model families)	This week is Week 4′s bargain taken to its limit. Chapter 14 is Week 4′s equivariance argument with a symmetry from physics rather than convenience; Chapter 23 is Week 4′s “pre-centred digits” honesty result made sharper.
Weeks 7 to 9 (LLMs, scale-free, lean)	Chapter 5′s data-regime argument is the bridge: this week is the extreme data-scarce end of the same curve those weeks explore at the other end.
Week 12 (project)	Chapter 26′s seven questions are the methods section of your critical brief; Chapter 24′s three questions are how you defend the choice.
PINN Sandbox	Chapters 8 to 12 as a manipulable object: it trains a real physics-informed network in the browser, with its own autodiff engine.
Symmetry Explorer	Chapters 14 and 15: rotate a molecule and watch the distances stay identical while the coordinates all change.
Diffusion and Thermodynamics Explorer	Chapters 20 and 21: the forward process dissolving structure into equilibrium, the reverse process recovering it, and the exact score field.

End of the Week 5 Reference Booklet. The equation was known everywhere.
Only one of the models was told.